FirstKnock Business Scraper & Contact Database — Complete Technical Blueprint

Untitled

A complete, self-contained technical reference for building a production-grade business scraper and contact database from scratch. Covers scraping strategy, anti-bot evasion, email/phone discovery, database schema, build sequence, and all working code implementations. Intended for a proficient Python developer building a self-hosted, zero-subscription lead generation system.

Table of Contents

Section 1 — Scraping at Scale

1A. Source Strategy

Google Maps (Playwright / Headless Chromium)

Approach: Playwright async with stealth patching. Google Maps renders via XHR/JSON internally

but the public-facing UI is fully JS-rendered. Direct API is rate-limited and requires billing. Scraping the rendered DOM is the only zero-cost path.

Core Selectors (verified 2024–2025)

Search results listRESULTS_CONTAINER= 'div[role="feed"]'RESULT_ITEM = 'div[role="feed"] > div > div > a'# Detail panel (after clicking a result)BUSINESS_NAME = 'h1.DUwDvf'RATING = 'div.F7nice > span[aria-hidden="true"]'REVIEW_COUNT = 'div.F7nice > span > span[aria-label]' CATEGORY = 'button.DkEaL'ADDRESS = 'button[data-item-id="address"] div.Io6YTePHONE = 'button[data-item-id^="phone"] div.Io6YTeWEBSITE = 'a[data-item-id="authority"] div.Io6YTe' HOURS_BUTTON = 'div[data-hide-tooltip-on-mouse-move] > div > div'PLUS_CODE = 'button[data-item-id="oloc"] div.Io6YTe'

Extractable Fields

Business name, address, phone, website URL, rating, review count, category, lat/lng (from URL), hours, plus code, photos count, owner response presence.

Rate Limits / Detection Thresholds

- Google aggressively fingerprints: canvas, WebGL, font enumeration, navigator.webdriver
- Soft block at ~150–200 requests/IP/hour without stealth
- Hard CAPTCHA at ~50 requests/IP/hour on datacenter IPs
- Residential proxies: ~300–500 req/IP/day before soft throttle
- Session rotation every 50–75 requests is safe

Core Scrape Loop

import asyncioimport randomfrom playwright.async_api import async_playwrightfrom urllib.parse import quoteSEARCH_URL = "https://www.google.com/maps/search/{query}/@{lat},{lng},13z" async def scrape_gmaps_zip(query: str, zip_code: str, proxy: dict) -> list[dict]:"""Scrape Google Maps for a query+ZIP. Returns list of business dicts.proxy: {"server": "http://host:port", "username": "u", "password": "p"}"""results = []async with async_playwright() as p:browser = await p.chromium.launch(headless=True,proxy=proxy,args=["--disable-blink-features=AutomationControlled","--no-sandbox","--disable-dev-shm-usage", "--disable-gpu","--window-size=1366,768",])context = await browser.new_context(viewport={"width": 1366, "height": 768},user_agent=get_random_ua(),locale="en-US", timezone_id="America/Chicago",geolocation={"latitude": 41.85, "longitude": -87.65}, permissions=["geolocation"],)await context.add_init_script("""Object.defineProperty(navigator, 'webdriver', {get: () => undefined});Object.defineProperty(navigator, 'plugins', {get: () => [1,2,3,4,5]});Object.defineProperty(navigator, 'languages', {get: () => ['en-US','en']});window.chrome = {runtime: {}};""")page = await context.new_page()search_query = quote(f"{query} {zip_code}") await page.goto(f"https://www.google.com/maps/search/{search_query}",wait_until="networkidle") await asyncio.sleep(random.uniform(2.0, 4.0))feed = page.locator('div[role="feed"]')prev_count = 0

for _ in range(15):items = await page.locator('div[role="feed"] > div > div > a').all()if len(items) == prev_count:breakprev_count = len(items)await feed.evaluate("el => el.scrollBy(0, 800)")await asyncio.sleep(random.uniform(1.2, 2.5))links = await page.locator('div[role="feed"] > div > div > a').evaluate_all("els => els.map(e => e.href)")for link in links[:20]:try:detail = await scrape_gmaps_detail(context, link)if detail:results.append(detail)await asyncio.sleep(random.uniform(1.5, 3.5))except Exception as e:continueawait browser.close()return resultsasync def scrape_gmaps_detail(context, url: str) -> dict | None:page = await context.new_page()try:await page.goto(url, wait_until="networkidle", timeout=30000)await asyncio.sleep(random.uniform(1.0, 2.0))name = await page.locator('h1.DUwDvf').first.inner_text()phone = ""website = ""address = ""category = ""try:phone = await page.locator('button[data-item-id^="phone"] div.Io6YTe').first.inner_text()except: passtry:website = await page.locator('a[data-item-id="authority"] div.Io6YTe').first.inner_text()except: passtry:address = await page.locator('button[data-item-id="address"] div.Io6YTe').first.inner_text()except: passtry:category = await page.locator('button.DkEaL').first.inner_text()except: passlat, lng = extract_latlng(page.url)return {"name": name.strip(),"phone": phone.strip(),"website": website.strip(),"address": address.strip(),"category": category.strip(),"lat": lat,"lng": lng,"source_url": url,"source": "google_maps",}except Exception as e:return Nonefinally:await page.close()def extract_latlng(url: str):import rem = re.search(r'@(-?\d+\.\d+),(-?\d+\.\d+)', url)if m:return float(m.group(1)), float(m.group(2))return None, None

Yelp

Approach: requests + BeautifulSoup. Yelp's search results are server-side rendered (SSR) with embedded JSON-LD and a __NEXT_DATA__ script tag containing structured business data. This is far more reliable than CSS scraping.

import requestsfrom bs4 import BeautifulSoupimport json, reHEADERS = {"User-Agent": get_random_ua(),"Accept-Language": "en-US,en;q=0.9","Accept": "text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8","Referer": "https://www.yelp.com/",}def scrape_yelp_search(category: str, location: str, session: requests.Session) -> list[dict]:""" category: e.g. 'pest-control', 'solar-installation', 'roofing'location: ZIP code string"""results = []for offset in range(0, 240, 30): # Yelp caps at 240 results per searchurl = f"https://www.yelp.com/search?find_desc={category}&find_loc={location}&start={offset}"resp = session.get(url, headers=HEADERS, timeout=15)if resp.status_code == 429:raise RateLimitError("Yelp 429")soup = BeautifulSoup(resp.text, "lxml")script = soup.find("script", id="__NEXT_DATA__")if script:data = json.loads(script.string)try:biz_list = data["props"]["pageProps"]["searchPageProps"]["mainContentComponentsListProps"]for item in biz_list:if item.get("componentKey") == "bizListItem":biz = item.get("props", {}).get("bizListItemProps", {})results.append({"name": biz.get("name"),"phone": biz.get("phone"),"address": biz.get("formattedAddress"),"rating": biz.get("rating"),"review_count": biz.get("reviewCount"),"categories": [c["title"] for c in biz.get("categories", [])],"yelp_url": "https://www.yelp.com" + biz.get("businessUrl", ""),"website": biz.get("website", {}).get("href"),

"source": "yelp",})except (KeyError, TypeError):passif not results:for card in soup.select('div[data-testid="serp-ia-card"]'):name_el = card.select_one('a.css-19v1rkv')phone_el = card.select_one('p.css-1p9ibgf')addr_el = card.select_one('address')results.append({"name": name_el.text.strip() if name_el else None,"phone": phone_el.text.strip() if phone_el else None, "address": addr_el.text.strip() if addr_el else None,"source": "yelp",})import time, random time.sleep(random.uniform(3.0, 6.0))return results

Yelp Rate Limits: Blocks datacenter IPs almost immediately. Residential proxies required. 1 request/3–6s per IP. Rotate IP every 10–15 requests. Yelp uses Akamai Bot Manager — TLS fingerprinting is a factor; use curl_cffi or tls-client instead of requests for better bypass.

Yellow Pages (YP.com)

Approach: requests + BeautifulSoup. YP.com is largely SSR with minimal JS. Easier to scrape than Google/Yelp but lower data quality and more duplicates. Datacenter proxies work at 1 req/2–4s. No CAPTCHA observed below 100 req/IP/hour.

def scrape_yellowpages(category: str, location: str, session: requests.Session) -> list[dict]:""" category: e.g. 'pest+control', 'solar+energy', 'roofing+contractors'location: ZIP code"""results = []for page_num in range(1, 6): # YP shows ~30/page, cap at 5 pagesurl = f"https://www.yellowpages.com/search?search_terms={category}&geo_location_terms={location}&page={page_num}"resp = session.get(url, headers=HEADERS, timeout=15)soup = BeautifulSoup(resp.text, "lxml")listings = soup.select('div.result > div.info')if not listings:breakfor listing in listings:name_el = listing.select_one('a.business-name')phone_el = listing.select_one('div.phones.phone.primary')addr_el = listing.select_one('div.adr')website_el = listing.select_one('a.track-visit-website')cats_el = listing.select('div.categories a')results.append({ "name": name_el.text.strip() if name_el else None,"phone": phone_el.text.strip() if phone_el else None,"address": addr_el.text.strip() if addr_el else None,"website": website_el.get("href") if website_el else None,"categories": [c.text for c in cats_el],"yp_url": "https://www.yellowpages.com" + name_el.get("href", "") if name_el else None,"source": "yellowpages",})import time, random time.sleep(random.uniform(2.0, 4.0))return results

1B. Geographic Coverage Strategy

Why ZIP-Level Beats City/State-Level

Google Maps and Yelp cap results at 20 (Maps) and 240 (Yelp) per search query. A city-level search for 'pest control Chicago' returns 20 results — but Chicago has 60+ ZIP codes, each potentially containing 10–20 unique businesses. ZIP-level queries multiply coverage by 10–30x in dense metros. Additionally, ZIP-level allows precise deduplication (same business appearing in multiple ZIPs is caught by phone/address normalization), and enables geographic load distribution across proxy pools.

42,741US ZIP Codes	**256,446**Total Queries (6 verticals)	**~1.5M**Unique Businesses (after dedup)	**1.3%**Top Records Needed for 20k

Free ZIP Code Sources

1. US Census Bureau ZCTA file — https://www.census.gov/geographies/reference-files/time-series/geo/gazetteer-files.html— Download: 2023_Gaz_zcta_national.zip — Fields: GEOID (ZIP), ALAND, AWATER, INTPTLAT,INTPTLONG — 33,791 ZCTAs covering all populated areas
2. simplemaps.com/data/us-zips — Free tier: 33,000+ ZIPs with city, state, county, lat/lng, population, timezone — Direct CSV download, no auth required — URL: https://simplemaps.com/static/data/us-zips/1.82/basic/simplemaps_uszips_basicv1.82.zip

ZIP Loading Code

import pandas as pddef load_zip_codes(csv_path: str = "uszips.csv") -> pd.DataFrame:df = pd.read_csv(csv_path, dtype={"zip": str})df["zip"] = df["zip"].str.zfill(5)df = df[df["state_id"].isin(["AL","AK","AZ","AR","CA","CO","CT","DE","FL","GA","HI","ID","IL","IN", "IA","KS","KY","LA","ME","MD","MA","MI","MN","MS","MO","MT","NE","NV", "NH","NJ","NM","NY","NC","ND","OH","OK","OR","PA","RI","SC","SD","TN", "TX","UT","VT","VA","WA","WV","WI","WY","DC"])]return df[["zip", "city", "state_id", "lat", "lng", "population"]].reset_index(drop=True)

PostgreSQL-Backed Job Queue (SKIP LOCKED)

-- schema.sqlCREATE TABLE scrape_jobs (id BIGSERIAL PRIMARY KEY,zip_code CHAR(5) NOT NULL,vertical VARCHAR(50) NOT NULL,source VARCHAR(20) NOT NULL,status VARCHAR(20) NOT NULL DEFAULT'pending',attempts SMALLINT NOT NULL DEFAULT0, last_error TEXT,created_at TIMESTAMPTZ NOT NULL DEFAULTNOW(),started_at TIMESTAMPTZ,finished_at TIMESTAMPTZ,result_count INT,UNIQUE(zip_code, vertical, source)); CREATE INDEX idx_scrape_jobs_status ON scrape_jobs(status);CREATE INDEX idx_scrape_jobs_zip ON scrape_jobs(zip_code);

import asyncioimport asyncpgfrom dataclasses import dataclassfrom typing import Optional @dataclassclass ScrapeJob:id: intzip_code: strvertical: strsource: strclass PGJobQueue:def __init__(self, dsn: str):self.dsn = dsnself.pool: Optional[asyncpg.Pool] = Noneasync def connect(self):self.pool = await asyncpg.create_pool(self.dsn, min_size=2, max_size=10)async def seed_jobs(self, zips: list[str], verticals: list[str], sources: list[str]):"""Idempotent — skips existing rows due to UNIQUE constraint."""async with self.pool.acquire() as conn:await conn.executemany("""INSERT INTO scrape_jobs (zip_code, vertical, source)VALUES ($1, $2, $3)ON CONFLICT DO NOTHING""",[(z, v, s) for z in zips for v in verticals for s in sources])async def claim_job(self,

worker_id: str) -> Optional[ScrapeJob]:"""Atomically claim one pending job using SKIP LOCKED."""async with self.pool.acquire() as conn:row = await conn.fetchrow("""UPDATE scrape_jobsSET status = 'running', started_at = NOW(), attempts = attempts + 1WHERE id = (SELECT id FROM scrape_jobsWHERE status = 'pending'OR (status = 'failed' AND attempts < 3)ORDER BY idLIMIT 1FOR UPDATE SKIP LOCKED)RETURNING id, zip_code, vertical, source""")if row:return ScrapeJob(**dict(row))return Noneasync def complete_job(self, job_id: int, result_count: int):async with self.pool.acquire() as conn:await conn.execute("""UPDATE scrape_jobsSET status = 'done', finished_at = NOW(), result_count = $2WHERE id = $1""", job_id, result_count)async def fail_job(self, job_id: int, error: str):async with self.pool.acquire() as conn:await conn.execute(""" UPDATE scrape_jobsSET status = 'failed', finished_at = NOW(), last_error = $2WHERE id = $1""", job_id, error)async def stats(self) -> dict:async with self.pool.acquire() as conn:rows = await conn.fetch("""SELECT status, COUNT(*) as cnt FROM scrape_jobs GROUP BY status""")return {r["status"]: r["cnt"] for r in rows}

Worker Pool

async def worker(queue: PGJobQueue, scraper_fn, concurrency_sem: asyncio.Semaphore):while True:job = await queue.claim_job("worker")if not job:await asyncio.sleep(5)continueasync with concurrency_sem:try:proxy = proxy_manager.get_proxy()results = await scraper_fn(job.vertical, job.zip_code, proxy)await save_results(results)await queue.complete_job(job.id, len(results)) except Exception as e:await queue.fail_job(job.id, str(e))async def run_workers(dsn: str, n_workers: int = 10):queue = PGJobQueue(dsn)await queue.connect()sem = asyncio.Semaphore(n_workers)tasks = [asyncio.create_task(worker(queue, scrape_gmaps_zip, sem))for _ in range(n_workers)]await asyncio.gather(*tasks)

Realistic Time Estimates

Table 1 — Worker Concurrency vs. Throughput (assumes 4–6s avg per request including page load + parsing)

Concurrency	Req/hr	256K Jobs ETA	Notes
5 workers	~300	~35 days	Single residential IP pool, safe
20 workers	~1,200	~9 days	20+ residential proxies needed
50 workers	~3,000	~3.5 days	50+ proxies, monitor block rate
100 workers	~6,000	~1.8 days	Requires proxy rotation infra

Proxy Rotation Manager

import asyncioimport aiohttpimport randomimport timefrom dataclasses import dataclass, fieldfrom typing import Optionalimport logginglogger = logging.getLogger(__name__)@dataclassclass Proxy: server: strusername: str = ""password: str = ""failures: int = 0last_used: float = 0.0last_checked: float = 0.0alive: bool = Truerequests_made: int = 0def to_playwright_dict(self) -> dict:d = {"server": self.server}if self.username:d["username"] = self.usernamed["password"] = self.passwordreturn d def to_requests_dict(self) -> dict:if self.username:auth = f"{self.username}:{self.password}@"proto = self.server.split("://")[0]host = self.server.split("://")[1]url = f"{proto}://{auth}{host}"else:url = self.serverreturn {"http": url, "https": url}class ProxyManager:def __init__(self,proxies: list[Proxy], max_failures: int = 3,cooldown_seconds: int = 300,check_url: str = "https://httpbin.org/ip", min_delay_between_uses: float = 1.0,):self.proxies = proxiesself.max_failures = max_failures self.cooldown = cooldown_secondsself.check_url = check_urlself.min_delay = min_delay_between_usesself._lock = asyncio.Lock()@classmethoddef from_list(cls, proxy_strings: list[str], **kwargs) -> "ProxyManager":proxies = []for s in proxy_strings:if "@" in s:proto_auth, hostport = s.rsplit("@", 1)proto, auth = proto_auth.split("://", 1)user, pwd = auth.split(":", 1) proxies.append(Proxy(server=f"{proto}://{hostport}", username=user, password=pwd))else: proxies.append(Proxy(server=s))return cls(proxies, **kwargs)async def get_proxy(self) -> Optional[Proxy]:async with self._lock:now = time.time()candidates = [p for p in self.proxiesif p.alive and p.failures < self.max_failuresand (now - p.last_used) >= self.min_delay]if not candidates: revivable = [p for p in self.proxiesif not p.alive and (now - p.last_checked) > self.cooldown]if revivable:proxy = random.choice(revivable)proxy.alive = Trueproxy.failures = 0return proxyreturn Noneweights = [(self.max_failures - p.failures + 1) * (now - p.last_used + 1)for p in candidates] proxy = random.choices(candidates, weights=weights, k=1)[0]proxy.last_used = now proxy.requests_made += 1return proxyasync def mark_failure(self, proxy: Proxy, error: str = ""): async with self._lock:proxy.failures += 1proxy.last_checked = time.time()if proxy.failures >= self.max_failures:proxy.alive = Falselogger.warning(f"Proxy {proxy.server} marked dead after {proxy.failures} failures.")async def mark_success(self, proxy: Proxy):async with self._lock: proxy.failures = max(0, proxy.failures - 1)async def health_check_all(self, timeout: int = 10):async def check(proxy: Proxy):try:async with aiohttp.ClientSession() as session:async with session.get(self.check_url,proxy=proxy.server,proxy_auth=aiohttp.BasicAuth(proxy.username,proxy.password) if proxy.username else None,timeout=aiohttp.ClientTimeout(total=timeout)) as resp:if resp.status == 200:proxy.alive = Trueproxy.failures = 0else:proxy.alive = Falseexcept Exception:proxy.alive = Falseproxy.last_checked = time.time()await asyncio.gather(*[check(p) for p in self.proxies])def stats(self) -> dict:alive = sum(1 for p in self.proxies if p.alive)return {"total": len(self.proxies),"alive": alive,"dead": len(self.proxies) - alive,"total_requests": sum(p.requests_made for p in self.proxies),}

Proxy Source Recommendations

Table 2 — Proxy Source Recommendations by Use Case

Type	Provider	Price	Use Case
Residential (required for Google/Yelp)	Bright Data	~$8.4/GB	Google Maps, Yelp
Residential	Oxylabs	~$8/GB	Google Maps, Yelp
Residential	Smartproxy	~$7/GB	Google Maps, Yelp
Residential	IPRoyal	~$3/GB	Google Maps, Yelp
Datacenter (OK for YP.com)	Webshare.io	Free (10 proxies)	Yellow Pages
Datacenter (free, unreliable)	ProxyScrape.com	Free	Testing only — 70%+ failure rate

Weighted User Agent Pool (22 Strings)

import randomUSER_AGENTS = [# Chrome on Windows (~65% weight)("Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/131.0.0.0 Safari/537.36", 12),("Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/130.0.0.0 Safari/537.36", 10),("Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/129.0.0.0 Safari/537.36", 8),("Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/128.0.0.0 Safari/537.36", 6),("Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/127.0.0.0 Safari/537.36", 4),# Chrome on macOS("Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/131.0.0.0 Safari/537.36", 8),("Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/130.0.0.0 Safari/537.36", 6),("Mozilla/5.0 (Macintosh; Intel Mac OS X 14_7_1) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/131.0.0.0 Safari/537.36", 5),# Firefox on Windows("Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:132.0) Gecko/20100101 Firefox/132.0", 6),("Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:131.0) Gecko/20100101 Firefox/131.0", 4),("Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:130.0) Gecko/20100101 Firefox/130.0", 3),# Firefox on macOS("Mozilla/5.0 (Macintosh; Intel Mac OS X 14.7; rv:132.0) Gecko/20100101 Firefox/132.0", 4),("Mozilla/5.0 (Macintosh; Intel Mac OS X 14.7; rv:131.0) Gecko/20100101 Firefox/131.0", 3),# Safari on macOS("Mozilla/5.0 (Macintosh; Intel Mac OS X 14_7_1) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/18.0 Safari/605.1.15", 5),("Mozilla/5.0 (Macintosh; Intel Mac OS X 14_6) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/17.6 Safari/605.1.15", 3),# Edge on Windows("Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/131.0.0.0 Safari/537.36 Edg/131.0.0.0", 4),("Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/130.0.0.0 Safari/537.36 Edg/130.0.0.0", 3),# Chrome on Linux("Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/131.0.0.0 Safari/537.36", 2),

("Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/130.0.0.0 Safari/537.36", 2),# Chrome on Android("Mozilla/5.0 (Linux; Android 14; Pixel 8) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/131.0.0.6778.39 Mobile Safari/537.36", 3),("Mozilla/5.0 (Linux; Android 13; SM-S918B) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/131.0.0.0 Mobile Safari/537.36", 2),# Safari on iPhone("Mozilla/5.0 (iPhone; CPU iPhone OS 18_1 like Mac OS X) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/18.1 Mobile/15E148 Safari/604.1", 3),]_ua_strings = [ua for ua, _ in USER_AGENTS]_ua_weights = [w for _, w in USER_AGENTS]def get_random_ua() -> str:return random.choices(_ua_strings, weights=_ua_weights, k=1)[0]

HumanDelay Class (Truncated Normal Distribution)

import numpy as npimport asyncioclass HumanDelay:PROFILES = {"fast_reader": {"mean": 2.5, "std": 0.8, "min": 1.0, "max": 6.0},"normal_reader": {"mean": 4.5, "std": 1.5, "min": 2.0, "max": 10.0}, "slow_reader": {"mean": 8.0, "std": 2.5, "min": 4.0, "max": 20.0},"page_load": {"mean": 1.5, "std": 0.5, "min": 0.5, "max": 4.0},"between_pages": {"mean": 3.0, "std": 1.0, "min": 1.5, "max": 8.0},} @staticmethoddef sample(profile: str = "normal_reader") -> float:p = HumanDelay.PROFILES[profile]while True:val = np.random.normal(p["mean"], p["std"])if p["min"] <= val <= p["max"]:return float(val)@staticmethodasync def wait(profile: str = "normal_reader"): delay = HumanDelay.sample(profile)await asyncio.sleep(delay)@staticmethoddef add_micro_jitter(base_delay: float, jitter_pct: float = 0.15) -> float:jitter = base_delay * jitter_pct * np.random.uniform(-1, 1)return max(0.1, base_delay + jitter)

Playwright Stealth Configuration

Library Status: AtuboDad/playwright_stealth is partially broken with Playwright 1.45+. Use the maintained fork Mattwmaster58/playwright_stealth or switch to patchright (https://github.com/Kaliiiiiiiiii-Vinyzu/patchright-python) which patches Chromium at the binary level — the most robust option in 2025.

Option A: patchright (recommended — drop-in Playwright replacement)# pip install patchright && patchright install chromiumfrom patchright.async_api import async_playwright as patched_playwrightasync def create_stealth_browser(proxy: dict):async with patched_playwright() as p:browser = await p.chromium.launch(headless=True,proxy=proxy,args=["--disable-blink-features=AutomationControlled","--no-sandbox","--disable-setuid-sandbox", "--disable-dev-shm-usage","--disable-accelerated-2d-canvas","--no-first-run","--no-zygote", "--disable-gpu","--window-size=1366,768","--disable-extensions","--disable-background-networking", "--disable-default-apps","--disable-sync","--disable-translate","--hide-scrollbars", "--metrics-recording-only","--mute-audio","--safebrowsing-disable-auto-update",])return browser# Option B: Mattwmaster58 fork# pip install git+https://github.com/Mattwmaster58/playwright_stealth.gitfrom playwright_stealth import stealth_async, StealthConfigasync def apply_stealth(page, ua: str):config = StealthConfig(navigator_languages=True,navigator_user_agent=True,navigator_vendor=True,webdriver=True,

chrome_app=True, chrome_csi=True,chrome_load_times=True,chrome_runtime=True, iframe_content_window=True,media_codecs=True,outerdimensions=True,hairline=True,)await stealth_async(page, config)await page.set_extra_http_headers({"User-Agent": ua, "Accept-Language": "en-US,en;q=0.9","Accept-Encoding": "gzip, deflate, br","sec-ch-ua": '"Chromium";v="131", "Not_A Brand";v="24", "Google Chrome";v="131"',"sec-ch-ua-mobile": "?0", "sec-ch-ua-platform": '"Windows"',})

SessionManager (Cookie Persistence)

import jsonimport osfrom pathlib import Pathclass SessionManager:def __init__(self, storage_dir: str = "./sessions"):self.storage_dir = Path(storage_dir)self.storage_dir.mkdir(exist_ok=True)def _session_path(self, proxy_id: str, domain: str) -> Path:safe_id = proxy_id.replace(":", "_").replace("/", "_").replace("@", "_")return self.storage_dir / f"{safe_id}_{domain}.json"async def save_cookies(self, context, proxy_id: str, domain: str):cookies = await context.cookies()path = self._session_path(proxy_id, domain)with open(path, "w") as f:json.dump(cookies, f)async def load_cookies(self, context, proxy_id: str, domain: str) -> bool:path = self._session_path(proxy_id, domain)if not path.exists():return Falsewith open(path) as f:cookies = json.load(f)if cookies:await context.add_cookies(cookies)return Truereturn Falsedef clear_session(self, proxy_id: str, domain: str):path = self._session_path(proxy_id, domain)if path.exists():path.unlink()def session_age_hours(self, proxy_id: str, domain: str) -> float:path = self._session_path(proxy_id, domain)if not path.exists():return float("inf")return (time.time() - path.stat().st_mtime) / 3600

CaptchaHandler (2captcha Integration)

import asyncioimport aiohttpimport reclass CaptchaHandler:"""Detects and solves reCAPTCHA v2/v3 via 2captcha.Pricing: ~$0.5-$1.00 per 1000 reCAPTCHA v2 solves (~$0.001/captcha)."""RECAPTCHA_PATTERNS= [r'google\.com/recaptcha',r'grecaptcha\.execute',r'data-sitekey',r'g-recaptcha',]HCAPTCHA_PATTERNS= [r'hcaptcha\.com',r'data-hcaptcha-sitekey',]def __init__(self, api_key: str, timeout: int = 120):self.api_key = api_keyself.timeout = timeout self.base_url = "https://2captcha.com"async def detect_captcha(self, page) -> str | None:content = await page.content()for pattern in self.RECAPTCHA_PATTERNSif re.search(pattern, content, re.IGNORECASE):return "recaptcha"for pattern in self.HCAPTCHA_PATTERNSif re.search(pattern, content, re.IGNORECASE):return "hcaptcha"if await page.locator('img[src*="captcha"]').count() > 0:return "image"return Noneasync def get_sitekey(self, page) -> str | None:try:el = page.locator('[data-sitekey]').firstreturn await el.get_attribute("data-sitekey")except:content = await page.content()m = re.search(r'["\']sitekey["\']\s*:\s*["\']([^"\']+)["\']', content)if m:return m.group(1)return Noneasync def solve_recaptcha_v2(self, page_url: str, sitekey: str) -> str | None:async with aiohttp.ClientSession() as session:submit_resp = await session.post(f"{self.base_url}/in.php",data={"key": self.api_key, "method": "userrecaptcha","googlekey": sitekey,"pageurl": page_url,"json": 1,})submit_data = await submit_resp.json()if submit_data.get("status") != 1:raise ValueError(f"2captcha submit failed: {submit_data}")captcha_id = submit_data["request"]for _ in range(self.timeout // 5):await

asyncio.sleep(5) result_resp = await session.get(f"{self.base_url}/res.php",params={"key": self.api_key, "action": "get", "id": captcha_id, "json": 1})result_data = await result_resp.json()if result_data.get("status") == 1:return result_data["request"]if result_data.get("request") != "CAPCHA_NOT_READY"raise ValueError(f"2captcha error: {result_data}")return Noneasync def inject_recaptcha_token(self, page, token: str):await page.evaluate(f""" document.getElementById('g-recaptcha-response').innerHTML = '{token}';if (typeof ___grecaptcha_cfg !== 'undefined') {{Object.entries(___grecaptcha_cfg.clients).forEach(([key, client]) => {{const callback = client?.l?.callback || client?.callback;if (typeof callback === 'function') callback('{token}');}});}}""")

1D. Open-Source Google Maps Scrapers — GitHub Audit

Table 3 — Open-Source Google Maps Scraper GitHub Audit

Repo	Stars	Language	Approach	Fit Score	Verdict
gosom/google-maps-scraper	~3,500	Go	Playwright (Rod) + Go concurrency. 33+ fields. Docker/K8s. REST API. Email harvesting.	6/10	Reference only — Go, not Python. Study scrolling/pagination logic.
gaspa93/googlemaps-scraper	~499	Python	Selenium + ChromeDriver. Review scraping for specific POI URLs — not a search/discovery tool.	2/10	Skip — wrong use case. No search functionality.
D3vd/google_maps_scraper	N/A	Unknown	Repo appears deleted or never existed under this path as of 2025.	N/A	Does not exist — skip.
omkarcloud/google-maps-scraper	~2,600	Python + Node.js	Botasaurus framework. 50+ fields including emails, social profiles. Pro paid tier.	7/10	Fork/reference — strip Botasaurus, replace with PlaywrightScraper.

christivn/mapScraper	~200	Python	Pure Python + Playwright. Extracts place ID, name, category, address, contact, coordinates, ratings.	7/10	Fork — good skeleton. Add ProxyManager + stealth layer on top.
ssecgroup/google-maps-scraper-pro	Low	Python	Fault-tolerant, checkpoint-based recovery, zero RAM streaming, deduplication built-in.	6/10	Reference for checkpoint/recovery patterns.

Final Recommendation: Build your own using christivn/mapScraper as skeleton + omkarcloud field extraction patterns. None of the existing Python scrapers include production-grade proxy rotation + stealth + async job queue. You need tight integration with the email/phone discovery pipeline.

1E. Extractable Fields Per Source

Table 4 — Extractable Fields Per Source (20 fields times 3 sources)

Field	Google Maps	Yelp	Yellow Pages
Business Name	Reliable	Reliable	Reliable
Phone Number	Reliable	Reliable	Reliable
Street Address	Reliable	Reliable	Reliable
City/State/ZIP	Reliable	Reliable	Reliable
Website URL	Reliable	~70% present	~60% present
Business Category	Reliable	Reliable	Reliable
Rating (stars)	Reliable	Reliable	~50% present
Review Count	Reliable	Reliable	~40% present
Latitude/Longitude	From URL	Not exposed	Not exposed
Business Hours	Reliable	On detail page	~60% present
Owner Name	~30% (Q&A/posts)	~40% (Meet the Owner)	Rare

Email Address	Not on Maps	Not on Yelp	Not on YP
Social Media Links	Not on Maps	~20%	Rare
Years in Business		~30%	~40%
Number of Employees			~20%
License/Accreditation		~25%	~30%
Photos Count	Reliable	Reliable	
Plus Code	Reliable		
Price Range	~40%	Reliable	
Claimed Status	Reliable	Reliable	~50%

Key Insight: No source exposes email directly. Email discovery requires the website URL rightarrow Section 2 pipeline. Phone is the most reliably extractable contact field across all three sources.

Section 2 — Email & Phone Discovery Engine

2A. Full Discovery Stack (Waterfall Approach)

The waterfall runs methods in order of reliability/cost. Stop as soon as a verified email is found.

Table 5 — Email Discovery Waterfall Stack

Priority	Method	Hit Rate	Notes
1	Website contact page crawler	~35%	Highest reliability, zero cost
2	SMTP pattern generation + verify	~25% additional	Requires domain + optional owner name
3	WHOIS registrant lookup	~8% additional	Many domains use privacy protection
4	Social media profile scraping	~5% additional	Facebook About pages; no direct LinkedIn
5	DuckDuckGo search discovery	~7% additional	No API key required; 1 req/2s rate limit
Total	All methods combined	~80% coverage	On businesses with websites

```
import reimport asyncioimport aiohttpfrom bs4 import BeautifulSoupfrom urllib.parse import urljoin, urlparsefrom typing import OptionalEMAIL_REGEX = re.compile( r'\b[A-Za-z0-9._%+\-]+@[A-Za-z0-9.\-]+\.[A-Za-z]{2,}\b')CONTACT_PAGE_PATHS = ["/contact", "/contact-us", "/contact_us", "/contactus","/about", "/about-us", "/about_us","/team", "/our-team", "/staff","/reach-us", "/get-in-touch","/info", "/support", "/help","/", # Homepage last] EXCLUDED_EMAIL_DOMAINS = {"example.com", "test.com", "domain.com", "email.com", "sentry.io", "sentry-next.io", "wixpress.com", "squarespace.com","wordpress.com", "godaddy.com", "amazonaws.com", "cloudflare.com","google.com", "facebook.com", "twitter.com", "instagram.com", "yelp.com", "yellowpages.com", "bbb.org", "angieslist.com",}GENERIC_LOCAL_PARTS = {"info", "contact", "hello", "admin", "support", "sales","office", "mail", "email", "service", "help", "team", "inquiry", "inquiries", "quote", "estimates",}async def crawl_website_for_emails(website_url: str, session: aiohttp.ClientSession,timeout: int = 10,max_pages: int = 6,) -> list[dict]:if not website_url.startswith(("http://", "https://")):website_url = "https://" + website_urlbase_domain = urlparse(website_url).netloc.lower().lstrip("www.")found_emails: dict[str, dict] = {}visited_urls: set[str] = set()headers = {"User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/131.0.0.0 Safari/537.36","Accept": "text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8","Accept-Language": "en-US,en;q=0.9",}async def fetch_page(url: str) -> Optional[str]:if url in visited_urls:return None visited_urls.add(url)try:async with session.get(url, headers=headers, timeout=aiohttp.ClientTimeout(total=timeout),allow_redirects=True, ssl=False) as resp:if resp.status == 200 and "text/html" in resp.headers.get("content-type", ""):return await resp.text(errors="replace")except Exception:passreturn Nonedef extract_emails_from_html(html: str, source_url: str) -> list[dict]:results = []soup = BeautifulSoup(html, "lxml")for a in soup.find_all("a", href=True):href = a["href"]if href.startswith("mailto:"):email = href[7:].split("?")[0].strip().lower()if is_valid_email(email, base_domain):results.append({"email": email,"source_url": source_url,"discovery_method": "mailto_link","confidence": 0.95,})text = soup.get_text(separator=" ")for match in EMAIL_REGEX.finditer(text):email = match.group(0).lower().strip(".")if is_valid_email(email, base_domain) and email not in {r["email"] for r in results}:local_part = email.split("@")[0]confidence = 0.85 if local_part in GENERIC_LOCAL_PARTSelse 0.90results.append({"email": email,"source_url": source_url, "discovery_method": "regex_text","confidence": confidence,})for match in EMAIL_REGEX.finditer(html):email = match.group(0).lower().strip(".")if is_valid_email(email, base_domain) and email not in {r["email"] for r in results}:results.append({"email": email, "source_url": source_url,"discovery_method": "regex_html","confidence": 0.80,})return resultsdef is_valid_email(email: str, site_domain: str) -> bool:if not EMAIL_REGEX.match(email):return False domain = email.split("@")[1].lower()if domain in EXCLUDED_EMAIL_DOMAINS:return Falseif len(email) > 100 or len(email) < 6:return Falseif any(x in email for x in ["@2x", ".png", ".jpg", ".gif", ".svg", ".css", ".js"]):return Falsereturn Truepages_checked = 0for path in CONTACT_PAGE_PATHSif pages_checked >= max_pages:breakurl = urljoin(website_url, path)
```

html = await fetch_page(url)if html:emails = extract_emails_from_html(html, url)for e in emails:if e["email"] not in found_emails:found_emails[e["email"]] = epages_checked += 1if found_emails and pages_checked >= 2:breakawait asyncio.sleep(0.3)if not found_emails and pages_checked > 0: homepage_html = await fetch_page(website_url)if homepage_html:soup = BeautifulSoup(homepage_html, "lxml")internal_links = []for a in soup.find_all("a", href=True):href = urljoin(website_url, a["href"])parsed = urlparse(href)if parsed.netloc.lstrip("www.") == base_domain and href not in visited_urls:link_text = a.get_text().lower()if any(kw in link_text or kw in href.lower() for kw in ["contact", "about", "team", "reach", "touch"]):internal_links.insert(0, href)else: internal_links.append(href)for link in internal_links[:4]:html = await fetch_page(link)if html:emails = extract_emails_from_html(html, link)for e in emails:if e["email"] not in found_emails: found_emails[e["email"]] = eawait asyncio.sleep(0.3)return list(found_emails.values())

Method 2: SMTP Pattern Generation

def generate_email_candidates(owner_first: str,owner_last: str,business_name: str,domain: str,) -> list[tuple[str, float]]:f = owner_first.lower().strip()l = owner_last.lower().strip()f1 = f[0] if f else ""l1 = l[0] if l else ""biz = re.sub(r'[^a-z0-9]', '', business_name.lower())[:20]candidates = []if f and l: candidates += [(f"{f}@{domain}", 0.18),(f"{f}.{l}@{domain}", 0.22),(f"{f}{l}@{domain}", 0.12), (f"{f1}{l}@{domain}", 0.15),(f"{f1}.{l}@{domain}", 0.10),(f"{l}@{domain}", 0.08),(f"{l}.{f}@{domain}", 0.06),(f"{l}{f1}@{domain}", 0.05),]candidates += [(f"info@{domain}", 0.35),(f"contact@{domain}", 0.20),(f"office@{domain}", 0.15),(f"hello@{domain}", 0.12),(f"admin@{domain}", 0.10), (f"owner@{domain}", 0.08),(f"sales@{domain}", 0.08),(f"service@{domain}", 0.07), (f"support@{domain}", 0.06),(f"mail@{domain}", 0.05),]if biz:candidates += [(f"{biz}@{domain}", 0.06),(f"info@{biz}.com", 0.04),]seen = set()result = []for email, prob in candidates:if email not in seen and "@" in email:seen.add(email)result.append((email, prob))return result

SMTP Pattern Failure Modes

- Catch-all domains — server accepts RCPT TO for any address. Detection: send to definitely-does-not-exist-xyz123@domain — if accepted, it's catch-all.
- Greylisting — server temporarily rejects (451). Mitigation: retry after 5 minutes.
- Connection refused — no MX record or firewall blocks port 25. Mitigation: skip domain, mark as unverifiable.
- SMTP banner blocking — server rejects based on reverse DNS of connecting IP. Mitigation: use a clean IP with proper rDNS, or use a VPS with PTR record set.
- Rate limiting — server blocks after 3–5 RCPT TO attempts. Mitigation: max 3 attempts/domain/hour, spread across time.

Method 3: WHOIS Lookup

import whois # pip install python-whoisdef whois_email_lookup(domain: str) -> list[dict]:results = [] try:w = whois.whois(domain)emails = w.emails if isinstance(w.emails, list) else ([w.emails] if w.emails else [])for email in emails:if email and "@" in email:email = email.lower().strip()

privacy_indicators = ["whoisguard", "privacyprotect", "domainsbyproxy","contactprivacy", "withheldforprivacy", "registrar-abuse","abuse@", "noreply@", "privacy@",]if not any(p in email for p in privacy_indicators):results.append({"email": email,"source": "whois","confidence": 0.70, "registrant_name": str(w.name or ""),"registrant_org": str(w.org or ""),})except Exception:passreturn results

WHOIS Limitation: GDPR/CCPA compliance has pushed most registrars to redact WHOIS data. GoDaddy, Namecheap, Google Domains all use privacy protection by default. Effective on older domains (pre-2018) and some budget registrars. Registrant name from WHOIS is often the owner's real name — valuable even when email is redacted.

Method 4: Social Media Profile Scraping

async def scrape_facebook_about(business_name: str,session: aiohttp.ClientSession,) -> list[dict]: results = []from duckduckgo_search import DDGSwith DDGS() as ddgs:search_results = list(ddgs.text(f'site:facebook.com "{business_name}" about',max_results=3))fb_urls = [r["href"] for r in search_results if "facebook.com" in r.get("href", "")]if not fb_urls:return resultsfor fb_url in fb_urls[:2]:about_url = fb_url.rstrip("/") + "/about" if "/about" not in fb_url else fb_urltry:async with session.get(about_url, timeout=aiohttp.ClientTimeout(total=10)) as resp:html = await resp.text(errors="replace")emails = EMAIL_REGEX.findall(html)for email in emails:email = email.lower()if "@" in email and "facebook.com" not in email:results.append({"email": email, "source": "facebook_about","confidence": 0.75,})except Exception:passreturn results

LinkedIn Strategy: Do NOT attempt to scrape LinkedIn directly — IP bans are permanent. Practical approach: Google/DuckDuckGo search site:linkedin.com/in 'company name' 'pest control' 'owner OR founder OR president'. Extract name from search snippet, then use name for SMTP pattern generation.

Method 5: DuckDuckGo Search Email Discovery

from duckduckgo_search import DDGSimport reimport timedef ddg_email_search(business_name: str,domain: str = "",location: str = "",max_results: int = 10,) -> list[dict]:results = []queries = []if domain:queries.append(f'"{business_name}" email site:{domain}') queries.append(f'"{business_name}" contact "@{domain}"')queries.append(f'"{business_name}" {location} email contact')queries.append(f'"{business_name}" owner email')seen_emails = set()with DDGS() as ddgs:for query in queries[:3]:try:search_results = list(ddgs.text(query, max_results=max_results))for r in search_results:text = r.get("body", "") + " " + r.get("title", "")for match in EMAIL_REGEX.finditer(text):email = match.group(0).lower().strip(".")if email not in seen_emails and is_plausible_business_email(email):seen_emails.add(email)results.append({ "email": email,"source": "duckduckgo_search","source_url": r.get("href", ""),"confidence": 0.65,}) time.sleep(2.0)except Exception as e:if "202" in str(e) or "Ratelimit" in str(e):time.sleep(10)break return resultsdef is_plausible_business_email(email: str) -> bool:domain = email.split("@")[1] if "@" in email else ""excluded = {"example.com", "test.com", "sentry.io", "github.com","npmjs.com",

"pypi.org", "w3.org", "schema.org",}if domain in excluded:return Falseif len(email) > 80:return Falseif re.search(r'\.(png|jpg|gif|svg|css|js|php|html)$', email):return Falsereturn True

2B. Complete SMTP Verifier

import asyncioimport dns.resolverimport smtplibimport socketimport timeimport hashlibimport json import loggingfrom dataclasses import dataclass, fieldfrom enum import Enumfrom typing import Optionalfrom pathlib import Pathfrom collections import defaultdictlogger = logging.getLogger(__name__)class EmailStatus(Enum):VALID = "valid"INVALID = "invalid" CATCH_ALL = "catch_all"UNVERIFIABLE = "unverifiable"RATE_LIMITED = "rate_limited" GREYLISTED = "greylisted"@dataclassclass VerificationResult:email: strstatus: EmailStatus confidence_score: floatmethod: strmx_host: Optional[str] = Nonesmtp_response: Optional[str] = Noneis_catch_all: bool = Falseverified_at: float = field(default_factory=time.time)def to_dict(self) -> dict:return {"email": self.email,"status": self.status.value,"confidence_score": self.confidence_score, "method": self.method,"mx_host": self.mx_host,"smtp_response": self.smtp_response, "is_catch_all": self.is_catch_all,"verified_at": self.verified_at,}class SMTPVerifier:HELO_DOMAIN = "mail.firstknock.io"FROM_ADDRESS = "verify@firstknock.io"SMTP_TIMEOUT = 10 MAX_ATTEMPTS_PER_DOMAIN_PER_HOUR= 3CACHE_TTL_HOURS = 72def __init__(self, cache_path: str = "./smtp_cache.json"):self.cache_path = Path(cache_path)self._cache: dict[str, dict] = self._load_cache()self._domain_attempts: dict[str, list[float]] = defaultdict(list)self._lock = asyncio.Lock()self._mx_cache: dict[str, list[str]] = {}def _load_cache(self) -> dict:if self.cache_path.exists():try:with open(self.cache_path) as f:return json.load(f)except Exception: passreturn {}def _save_cache(self):with open(self.cache_path, "w") as f:json.dump(self._cache, f) def _cache_key(self, email: str) -> str:return hashlib.md5(email.lower().encode()).hexdigest()async def _check_rate_limit(self, domain: str) -> bool:async with self._lock:now = time.time()hour_ago = now - 3600self._domain_attempts[domain] = [t for t in self._domain_attempts[domain] if t > hour_ago]if len(self._domain_attempts[domain]) >= self.MAX_ATTEMPTS_PER_DOMAIN_PER_HOUR:return False self._domain_attempts[domain].append(now)return Trueasync def _resolve_mx(self, domain: str) -> list[str]:if domain in self._mx_cache:return self._mx_cache[domain]loop = asyncio.get_event_loop()try:records = await loop.run_in_executor(None,lambda: dns.resolver.resolve(domain, "MX"))mx_hosts = [str(r.exchange).rstrip(".") for r in sorted(records, key=lambda x: x.preference)]self._mx_cache[domain] = mx_hostsreturn mx_hostsexcept (dns.resolver.NXDOMAIN, dns.resolver.NoAnswer, dns.exception.DNSException): self._mx_cache[domain] = []return []def _smtp_verify_sync(self, email: str, mx_host: str, check_catch_all: bool = True) -> tuple:domain = email.split("@")[1]is_catch_all = Falsetry:with smtplib.SMTP(timeout=self.SMTP_TIMEOUT) as smtp:smtp.connect(mx_host, 25) smtp.helo(self.HELO_DOMAIN)smtp.mail(self.FROM_ADDRESS)if check_catch_all:canary = f"definitely-does-not-exist-xk7q2m@{domain}"code, msg = smtp.rcpt(canary)if code == 250: is_catch_all = Truecode, msg = smtp.rcpt(email)response = f"{code} {msg.decode('utf-8',

errors='replace')}" return EmailStatus.CATCH_ALL, response, True code, msg = smtp.rcpt(email) response = f"{code} {msg.decode('utf-8', errors='replace')}" if code == 250: return EmailStatus.VALID, response, is_catch_all elif code in (550, 551, 553, 554): return EmailStatus.INVALID, response, is_catch_all elif code == 451: return EmailStatus.GREYLISTED, response, is_catch_all else: return EmailStatus.UNVERIFIABLE, response, is_catch_all except smtplib.SMTPConnectError as e: return EmailStatus.UNVERIFIABLE, f"ConnectError: {e}", False except smtplib.SMTPServerDisconnected as e: return EmailStatus.UNVERIFIABLE, f"Disconnected: {e}", False except socket.timeout: return EmailStatus.UNVERIFIABLE, "Timeout", False except ConnectionRefusedError: return EmailStatus.UNVERIFIABLE, "ConnectionRefused", False except OSError as e: return EmailStatus.UNVERIFIABLE, f"OSError: {e}", False async def verify(self, email: str) -> VerificationResult: email = email.lower().strip() domain = email.split("@")[1] if "@" in email else "" if not domain: return VerificationResult(email=email, status=EmailStatus.INVALID, confidence_score=0.0, method="format_check") if not await self._check_rate_limit(domain): return VerificationResult(email=email, status=EmailStatus.RATE_LIMITED, confidence_score=0.0, method="rate_limited") mx_hosts = await self._resolve_mx(domain) if not mx_hosts: return VerificationResult(email=email, status=EmailStatus.UNVERIFIABLE, confidence_score=0.1, method="no_mx_record") loop = asyncio.get_event_loop() mx_host = mx_hosts[0] try: status, smtp_response, is_catch_all = await loop.run_in_executor(None, lambda: self._smtp_verify_sync(email, mx_host, check_catch_all=True)) except Exception as e: status = EmailStatus.UNVERIFIABLE smtp_response = str(e) is_catch_all = False confidence = self._compute_confidence(status, is_catch_all, email) return VerificationResult(email=email, status=status, confidence_score=confidence, method="smtp_rcpt_to", mx_host=mx_host, smtp_response=smtp_response[:200] if smtp_response else None, is_catch_all=is_catch_all,) def _compute_confidence(self, status: EmailStatus, is_catch_all: bool, email: str) -> float: base_scores = { EmailStatus.VALID: 0.92, EmailStatus.CATCH_ALL: 0.45, EmailStatus.INVALID: 0.02, EmailStatus.UNVERIFIABLE: 0.30, EmailStatus.RATE_LIMITED: 0.40, EmailStatus.GREYLISTED: 0.55, } score = base_scores.get(status, 0.30) domain = email.split("@")[1] generic_domains = {"gmail.com", "yahoo.com", "hotmail.com", "outlook.com", "aol.com"} if domain not in generic_domains: score = min(1.0, score + 0.05) local = email.split("@")[0] if local in {"info", "contact", "admin", "support"}: score = max(0.0, score - 0.03) return round(score, 3) async def verify_batch(self, emails: list[str], concurrency: int = 5) -> list[VerificationResult]: sem = asyncio.Semaphore(concurrency) async def verify_one(email: str) -> VerificationResult: async with sem: result = await self.verify(email) await asyncio.sleep(0.5) return result return await asyncio.gather(*[verify_one(e) for e in emails]) def close(self): self._save_cache()

2C. Phone Number Extraction & Validation

import phonenumbers from phonenumbers import PhoneNumberFormat, NumberParseException import re from typing import Optional def extract_phones_from_text(text: str, default_region: str = "US") -> list[dict]: results = [] seen_e164 = set() for match in

```
phonenumbers.PhoneNumberMatcher(text, default_region):try:num = match.numberif phonenumbers.is_valid_number(num):e164 = phonenumbers.format_number(num, PhoneNumberFormat.E164)if e164 not in seen_e164:seen_e164.add(e164)national = phonenumbers.format_number(num, PhoneNumberFormat.NATIONAL)results.append({"e164": e164,"national": national,"raw": match.raw_string,"number_type": phonenumbers.number_type(num),"is_mobile": phonenumbers.number_type(num) == phonenumbers.PhoneNumberType.MOBILE,"region": phonenumbers.region_code_for_number(num),})except Exception:continuereturn resultsdef normalize_phone(raw: str, default_region: str = "US") -> Optional[str]:try:num = phonenumbers.parse(raw, default_region)if phonenumbers.is_valid_number(num):return phonenumbers.format_number(num, PhoneNumberFormat.E164)except NumberParseException: passreturn Nonedef deduplicate_phones(phone_list: list[dict]) -> list[dict]:seen: dict[str, dict] = {} source_priority = {"google_maps": 3, "yelp": 2, "yellowpages": 1, "website": 2}for phone in phone_list:e164 = phone.get("e164")if not e164:e164 = normalize_phone(phone.get("raw", ""))if not e164:continueif e164 not in seen:seen[e164] = phone.copy()seen[e164]["sources"] = [phone.get("source", "unknown")]else:existing = seen[e164]src = phone.get("source", "unknown")if src not in existing["sources"]:existing["sources"].append(src)if source_priority.get(src, 0) > source_priority.get(existing.get("source",""), 0):seen[e164].update(phone)seen[e164]["sources"] = existing["sources"]return list(seen.values())class PhoneValidator:TOLL_FREE_PREFIXES = {"800", "833", "844", "855", "866", "877", "888"}PREMIUM_PREFIXES = {"900", "976"} @staticmethoddef classify(e164: str) -> dict:try:num = phonenumbers.parse(e164, "US")num_type = phonenumbers.number_type(num)national = phonenumbers.format_number(num, PhoneNumberFormat.NATIONAL)area_code = national.replace("(", "").split(")")[0].strip()return { "e164": e164,"national": national,"area_code": area_code,"is_mobile": num_type == phonenumbers.PhoneNumberType.MOBILE,"is_voip": num_type == phonenumbers.PhoneNumberType.VOIP,"is_toll_free": area_code in PhoneValidator.TOLL_FREE_PREFIXES,"is_premium": area_code in PhoneValidator.PREMIUM_PREFIXES,"is_valid": phonenumbers.is_valid_number(num),}except Exception as e:return {"e164": e164, "is_valid": False, "error": str(e)}
```

2D. Owner Name Extraction

Table 6 — Owner Name Source Confidence Reference

Source	Confidence	Notes
json_ld_schema (website)	0.85	Structured data — most reliable
google_maps_owner_field	0.80	Direct from Maps listing
yelp_meet_owner	0.75	Yelp 'Meet the Owner' section

website_about (regex)	0.70	Regex from About page text
linkedin_search	0.65	Inferred from search snippet
whois_registrant	0.60	Often privacy-protected
corroboration_bonus	+0.10 per source	Max +0.20 for multiple confirmations

from dataclasses import dataclass, fieldfrom typing import Optionalimport refrom bs4 import BeautifulSoup@dataclassclass OwnerNameResult:first_name: strlast_name: strfull_name: str source: strconfidence: floatraw_text: str = ""@propertydef email_local_parts(self) -> list[str]:f = self.first_name.lower()l = self.last_name.lower()f1 = f[0] if f else ""return [f"{f}.{l}", f"{f}{l}", f"{f1}{l}", f"{f1}.{l}",f"{f}", f"{l}", f"{l}.{f}", f"{l}{f1}",]OWNER_TITLE_PATTERNS= [r'(?:owner|founder|president|ceo|principal|proprietor|managing (?:director|partner))' r'\s*[:\-\u2013\u2014]?\s*([A-Z][a-z]+(?: [A-Z][a-z]+){1,3})', r'([A-Z][a-z]+(?: [A-Z][a-z]+){1,2})\s*[,\-\u2013]?\s*' r'(?:owner|founder|president|ceo|principal)',] NAME_BLACKLIST = {"Contact Us", "About Us", "Our Team", "Meet The", "Learn More","Read More", "Click Here", "Get Started", "Free Quote",}def extract_owner_from_website(html: str, url: str) -> list[OwnerNameResult]:results = []soup = BeautifulSoup(html, "lxml")text = soup.get_text(separator=" ", strip=True)for pattern in OWNER_TITLE_PATTERNSfor match in re.finditer(pattern, text, re.IGNORECASE):name = match.group(1).strip()if name in NAME_BLACKLIST or len(name) < 4:continueparts = name.split()if len(parts) >= 2: results.append(OwnerNameResult(first_name=parts[0],last_name=parts[-1],full_name=name, source="website_about",confidence=0.70,raw_text=match.group(0),))for script in soup.find_all("script", type="application/ld+json"):try:import jsondata = json.loads(script.string)if isinstance(data, list):data = data[0]for key in ["founder", "employee", "contactPoint"]:person = data.get(key, {})if isinstance(person, dict) and person.get("name"):name = person["name"]parts = name.split()if len(parts) >= 2:results.append(OwnerNameResult(first_name=parts[0], last_name=parts[-1],full_name=name,source="json_ld_schema",confidence=0.85,))except Exception:passreturn resultsdef score_owner_name_candidates(candidates: list[OwnerNameResult]) -> OwnerNameResult | None:if not candidates:return Nonename_groups: dict[str, list[OwnerNameResult]] = {}for c in candidates:key = c.full_name.lower().strip() name_groups.setdefault(key, []).append(c)scored = []for name_key, group in name_groups.items(): base_confidence = max(c.confidence for c in group)corroboration_bonus = min(0.20, (len(group) - 1) * 0.10)final_score = min(1.0, base_confidence + corroboration_bonus)best = max(group, key=lambda x: x.confidence)best.confidence = final_scorescored.append(best)return max(scored, key=lambda x: x.confidence)

Section 3 — Data Storage & Schema

Decision: Use PostgreSQL. Full stop. SQLite is a non-starter for this workload: no true concurrent writes, no SELECT FOR UPDATE SKIP LOCKED, no pg_trgm trigram indexes for fuzzy deduplication, no LISTEN/NOTIFY for worker coordination. Breaks under 10+ concurrent scraper workers hammering inserts.

docker-compose.yml

version: "3.9"services:postgres:image: postgres:16-alpinecontainer_name: firstknock_dbrestart: unless-stoppedenvironment:POSTGRES_USER: ${DB_USER:-firstknock} POSTGRES_PASSWORD: ${DB_PASSWORD:-changeme}POSTGRES_DB: ${DB_NAME:-firstknock}volumes:- postgres_data:/var/lib/postgresql/data- ./database/schema.sql:/docker-entrypoint-initdb.d/01_schema.sqlports:- "5432:5432"healthcheck: test: ["CMD-SHELL", "pg_isready -U ${DB_USER:-firstknock}"]interval: 10stimeout: 5sretries: 5 pgadmin:image: dpage/pgadmin4:latestcontainer_name: firstknock_pgadminrestart: unless-stopped environment:PGADMIN_DEFAULT_EMAIL:${PGADMIN_EMAIL:-admin@firstknock.local} PGADMIN_DEFAULT_PASSWORD:${PGADMIN_PASSWORD:-admin} PGADMIN_CONFIG_SERVER_MODE: "False" PGADMIN_CONFIG_MASTER_PASSWORD_REQUIRED:"False"volumes:- pgadmin_data:/var/lib/pgadminports:- "5050:80"depends_on:postgres:condition: service_healthy volumes:postgres_data:pgadmin_data:

3B. Complete Database Schema

-- database/schema.sqlCREATE EXTENSION IF NOT EXISTS "uuid-ossp";CREATE EXTENSION IF NOT EXISTS "pg_trgm";CREATE EXTENSION IF NOT EXISTS "btree_gin";-- ENUMSCREATE TYPE scrape_status_enum AS ENUM ('pending', 'in_progress', 'completed', 'failed', 'skipped'); CREATE TYPE email_status_enum AS ENUM ('unknown', 'valid', 'invalid', 'catch_all', 'disposable', 'role_based', 'unverifiable', 'bounced');CREATE TYPE phone_status_enum AS ENUM ('unknown', 'valid', 'invalid', 'disconnected', 'voip', 'mobile', 'landline');CREATE TYPE outreach_status_enum AS ENUM ('not_contacted', 'queued', 'emailed', 'called', 'replied','interested', 'not_interested', 'unsubscribed', 'bounced', 'converted');CREATE TYPE job_status_enum AS ENUM ('pending', 'claimed', 'running', 'completed', 'failed', 'dead');CREATE TYPE duplicate_status_enum AS ENUM ('original', 'duplicate', 'merged');-- SOURCESCREATE TABLE sources (id SERIAL PRIMARY KEY, name VARCHAR(100) NOT NULL UNIQUE,base_url TEXT,is_active BOOLEAN NOT NULL DEFAULT TRUE,scrape_delay_ms INTEGER NOT NULL DEFAULT 2000,created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW());INSERT INTO sources (name, base_url, scrape_delay_ms) VALUES('google_maps', 'https://maps.google.com', 2500),('yelp', 'https://www.yelp.com', 1500),('yellow_pages', 'https://www.yellowpages.com', 1000),('website', NULL, 500),('manual', NULL, 0);-- BUSINESSES CREATE TABLE businesses (id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),name

```
VARCHAR(500) NOT NULL,name_normalized VARCHAR(500) GENERATED ALWAYS AS (
lower(regexp_replace(name, '[^a-zA-Z0-9]', '', 'g'))) STORED,vertical VARCHAR(100) NOT NULL,
sub_vertical VARCHAR(100),address_line1 VARCHAR(500),address_line2 VARCHAR(100),city
VARCHAR(200),state CHAR(2),zip_code VARCHAR(10),county VARCHAR(200),country CHAR(2)
NOT NULL DEFAULT'US',latitude NUMERIC(10, 7),longitude NUMERIC(10, 7),
address_normalized VARCHAR(1000) GENERATED ALWAYS AS (lower(regexp_replace(
COALESCE(address_line1,'') || ' ' ||COALESCE(city,'') || ' ' ||COALESCE(state,'') || ' ' ||
COALESCE(zip_code,''),'[^a-zA-Z0-9]', '', 'g'))) STORED,phone VARCHAR(20),phone_normalized
VARCHAR(15),phone_status phone_status_enum NOT NULL DEFAULT'unknown',phone_2
VARCHAR(20),website TEXT,website_domain VARCHAR(500),email VARCHAR(500),
email_status email_status_enum NOT NULL DEFAULT'unknown',email_2 VARCHAR(500),
description TEXT,categories TEXT[],rating NUMERIC(3, 1),review_count INTEGER,
years_in_business INTEGER,employee_count VARCHAR(50),is_franchise BOOLEAN,hours_json
JSONB,social_links JSONB,source_id INTEGER REFERENCES sources(id),source_listing_id
VARCHAR(500),source_url TEXT,all_source_ids INTEGER[] NOT NULL DEFAULT'{}',
scrape_status scrape_status_enum NOT NULL DEFAULT'pending',last_scraped_at
TIMESTAMPTZ,scrape_error TEXT,raw_data JSONB,outreach_status outreach_status_enum
NOT NULL DEFAULT'not_contacted',last_contacted_at TIMESTAMPTZ,next_follow_up_at
TIMESTAMPTZ,outreach_notes TEXT,duplicate_status duplicate_status_enum NOT NULL
DEFAULT'original',canonical_id UUID REFERENCES businesses(id),merged_ids UUID[] NOT
NULL DEFAULT'{}',deleted_at TIMESTAMPTZ,created_at TIMESTAMPTZ NOT NULL DEFAULT
NOW(),updated_at TIMESTAMPTZ NOT NULL DEFAULTNOW());CREATE INDEX
idx_businesses_vertical ON businesses(vertical) WHERE deleted_at IS NULL;CREATE INDEX
idx_businesses_state ON businesses(state) WHERE deleted_at IS NULL;CREATE INDEX
idx_businesses_zip ON businesses(zip_code) WHERE deleted_at IS NULL;CREATE INDEX
idx_businesses_vertical_state ON businesses(vertical, state) WHERE deleted_at IS NULL;
CREATE INDEX idx_businesses_outreach ON businesses(outreach_status) WHERE deleted_at
IS NULL;CREATE INDEX idx_businesses_email_status ON businesses(email_status) WHERE
deleted_at IS NULL;CREATE INDEX idx_businesses_phone_norm ON
businesses(phone_normalized) WHERE phone_normalized IS NOT NULL;CREATE INDEX
idx_businesses_website_domain ON businesses(website_domain) WHERE website_domain IS
NOT NULL;CREATE INDEX idx_businesses_name_trgm ON businesses USING GIN(name
gin_trgm_ops);CREATE INDEX idx_businesses_name_norm ON businesses(name_normalized);
CREATE INDEX idx_businesses_addr_norm ON businesses(address_normalized);CREATE
INDEX idx_businesses_canonical ON businesses(canonical_id) WHERE canonical_id IS NOT
NULL;CREATE INDEX idx_businesses_created ON businesses(created_at DESC);CREATE
INDEX idx_businesses_follow_up ON businesses(next_follow_up_at) WHERE next_follow_up_at
IS NOT NULL AND deleted_at IS NULL;CREATE INDEX idx_businesses_scrape_status ON
businesses(scrape_status) WHERE deleted_at IS NULL;-- CONTACTSCREATE TABLE contacts (
id UUID PRIMARY KEY DEFAULTuuid_generate_v4(),business_id UUID NOT NULL
```

```
REFERENCES businesses(id) ON DELETE CASCADE,first_name VARCHAR(200),last_name
VARCHAR(200),full_name VARCHAR(500),title VARCHAR(300),is_owner BOOLEAN NOT NULL
DEFAULTFALSE,is_decision_maker BOOLEAN NOT NULL DEFAULTFALSE,email
VARCHAR(500),email_status email_status_enum NOT NULL DEFAULT'unknown',email_source
VARCHAR(100),email_confidence NUMERIC(4,3),phone VARCHAR(20),phone_normalized
VARCHAR(15),phone_status phone_status_enum NOT NULL DEFAULT'unknown',linkedin_url
TEXT,outreach_status outreach_status_enum NOT NULL DEFAULT'not_contacted',
last_contacted_at TIMESTAMPTZ,next_follow_up_at TIMESTAMPTZ,deleted_at TIMESTAMPTZ,
created_at TIMESTAMPTZ NOT NULL DEFAULTNOW(),updated_at TIMESTAMPTZ NOT NULL
DEFAULTNOW());CREATE INDEX idx_contacts_business_id ON contacts(business_id);CREATE
INDEX idx_contacts_email ON contacts(email) WHERE email IS NOT NULL;CREATE INDEX
idx_contacts_email_status ON contacts(email_status);CREATE INDEX idx_contacts_outreach ON
contacts(outreach_status) WHERE deleted_at IS NULL;-- SCRAPE JOBSCREATE TABLE
scrape_jobs (id BIGSERIAL PRIMARY KEY,source_id INTEGER NOT NULL REFERENCES
sources(id),vertical VARCHAR(100) NOT NULL,search_query VARCHAR(500) NOT NULL,
zip_code VARCHAR(10),state CHAR(2),city VARCHAR(200),radius_miles INTEGER NOT NULL
DEFAULT25,status job_status_enum NOT NULL DEFAULT'pending',priority SMALLINT NOT
NULL DEFAULT5,attempts SMALLINT NOT NULL DEFAULT0,max_attempts SMALLINT NOT
NULL DEFAULT3,worker_id VARCHAR(100),claimed_at TIMESTAMPTZ,started_at
TIMESTAMPTZ,completed_at TIMESTAMPTZ,next_retry_at TIMESTAMPTZ,results_found
INTEGER,results_saved INTEGER,error_message TEXT,error_traceback TEXT,created_at
TIMESTAMPTZ NOT NULL DEFAULTNOW(),updated_at TIMESTAMPTZ NOT NULL DEFAULT
NOW());CREATE INDEX idx_jobs_status_priority ON scrape_jobs(priority, id)WHERE status =
'pending' AND (next_retry_at IS NULL OR next_retry_at <= NOW());CREATE INDEX
idx_jobs_worker ON scrape_jobs(worker_id) WHERE worker_id IS NOT NULL;CREATE INDEX
idx_jobs_vertical ON scrape_jobs(vertical);CREATE INDEX idx_jobs_zip ON
scrape_jobs(zip_code);CREATE INDEX idx_jobs_status ON scrape_jobs(status);-- EMAIL
VERIFICATION RESULTSCREATE TABLE email_verification_results (id BIGSERIAL PRIMARY
KEY,email VARCHAR(500) NOT NULL,business_id UUID REFERENCES businesses(id),
contact_id UUID REFERENCES contacts(id),syntax_valid BOOLEAN,mx_found BOOLEAN,
mx_records TEXT[],smtp_connectable BOOLEAN,smtp_accepted BOOLEAN,is_catch_all
BOOLEAN,is_disposable BOOLEAN,is_role_based BOOLEAN,final_status email_status_enum
NOT NULL,confidence NUMERIC(4,3),verified_at TIMESTAMPTZ NOT NULL DEFAULTNOW(),
verifier_version VARCHAR(20),smtp_response TEXT,smtp_code SMALLINT);CREATE UNIQUE
INDEX idx_email_verify_email ON email_verification_results(email);CREATE INDEX
idx_email_verify_business ON email_verification_results(business_id);CREATE INDEX
idx_email_verify_status ON email_verification_results(final_status);-- OUTREACH LOGCREATE
TABLE outreach_log (id BIGSERIAL PRIMARY KEY,business_id UUID REFERENCES
businesses(id),contact_id UUID REFERENCES contacts(id),channel VARCHAR(50) NOT NULL,
direction VARCHAR(10) NOT NULL DEFAULT'outbound',subject VARCHAR(500),body_preview
```

TEXT, status VARCHAR(50) NOT NULL,external_id VARCHAR(500),sent_at TIMESTAMPTZ, delivered_at TIMESTAMPTZ,opened_at TIMESTAMPTZ,replied_at TIMESTAMPTZ,created_at TIMESTAMPTZ NOT NULL DEFAULT NOW());CREATE INDEX idx_outreach_business ON outreach_log(business_id);CREATE INDEX idx_outreach_contact ON outreach_log(contact_id); CREATE INDEX idx_outreach_channel ON outreach_log(channel);CREATE INDEX idx_outreach_sent_at ON outreach_log(sent_at DESC);-- AUTO-UPDATE TRIGGERSCREATE OR REPLACE FUNCTION update_updated_at()RETURNS TRIGGER AS $$BEGIN NEW.updated_at = NOW();RETURN NEW;END;$$ LANGUAGE plpgsql;CREATE TRIGGER trg_businesses_updated_atBEFORE UPDATE ON businessesFOR EACH ROW EXECUTE FUNCTION update_updated_at();CREATE TRIGGER trg_contacts_updated_atBEFORE UPDATE ON contactsFOR EACH ROW EXECUTE FUNCTION update_updated_at();CREATE TRIGGER trg_scrape_jobs_updated_atBEFORE UPDATE ON scrape_jobsFOR EACH ROW EXECUTE FUNCTION update_updated_at();CREATE TRIGGER trg_sources_updated_atBEFORE UPDATE ON sourcesFOR EACH ROW EXECUTE FUNCTION update_updated_at();

3C. Deduplication Logic

Duplicate rules (in priority order): (1) Same phone_normalized rightarrow definite duplicate; (2) Same website_domain rightarrow definite duplicate; (3) name fuzzy >85% + same ZIP rightarrow likely duplicate (flagged); (4) Same address_normalized rightarrow likely duplicate (flagged).

from __future__ import annotationsimport reimport loggingfrom dataclasses import dataclass, field from typing import Optionalfrom urllib.parse import urlparsefrom rapidfuzz import fuzzfrom sqlalchemy import textfrom sqlalchemy.orm import Sessionlogger = logging.getLogger(__name__) FUZZY_NAME_THRESHOLD = 85FUZZY_ADDR_THRESHOLD = 90@dataclassclass DuplicateMatch:existing_id: strcandidate_id: strmatch_type: strconfidence: floatis_definite: bool @dataclassclass BusinessRecord:id: strname: strphone_normalized: Optional[str]website_domain: Optional[str]zip_code: Optional[str]address_normalized: Optional[str]name_normalized: Optional[str]email: Optional[str] = Nonephone: Optional[str] = Noneaddress_line1: Optional[str] = Nonerating: Optional[float] = Nonereview_count: Optional[int] = Noneall_source_ids: list = field(default_factory=list)merged_ids: list = field(default_factory=list)class Deduplicator:def __init__(self, session: Session):self.session = sessiondef find_duplicates(self, candidate: BusinessRecord) -> list[DuplicateMatch]:matches: list[DuplicateMatch] = []if candidate.phone_normalized:m = self._check_phone(candidate)if m: matches.append(m)if candidate.website_domain:m = self._check_domain(candidate)if m: matches.append(m)if candidate.zip_code and candidate.name_normalized: matches.extend(self._check_name_zip_fuzzy(candidate))if candidate.address_normalized: matches.extend(self._check_address(candidate))seen: dict[str, DuplicateMatch] = {}for m in matches:if m.existing_id not in seen or m.confidence > seen[m.existing_id].confidence: seen[m.existing_id] = mreturn list(seen.values())def merge_into_canonical(self, canonical_id: str, duplicate_id: str, session=None) -> None:s = session or self.sessioncanonical =

```
self._fetch_record(canonical_id, s)duplicate = self._fetch_record(duplicate_id, s)if not canonical or not duplicate:returnfillable_fields = ['email', 'email_status', 'phone', 'phone_normalized', 'phone_status','website', 'website_domain', 'address_line1', 'address_line2','city', 'state', 'zip_code', 'county', 'latitude', 'longitude','description', 'rating', 'review_count', 'years_in_business', 'employee_count', 'hours_json', 'social_links',]updates: dict = {}for f in fillable_fields:if canonical.get(f) is None and duplicate.get(f) is not None:updates[f] = duplicate[f] merged_source_ids = list(set((canonical.get('all_source_ids') or []) +(duplicate.get('all_source_ids') or [])))updates['all_source_ids'] = merged_source_idsmerged_ids = list(set( (canonical.get('merged_ids') or []) +[duplicate_id] +(duplicate.get('merged_ids') or []))) updates['merged_ids'] = merged_idsif updates:set_clauses = ', '.join(f"{k} = :{k}" for k in updates) updates['canonical_id'] = canonical_ids.execute(text(f"UPDATE businesses SET {set_clauses} WHERE id = :canonical_id"), updates)s.execute(text("""UPDATE businessesSET duplicate_status = 'duplicate',canonical_id = :canonical_id,deleted_at = NOW()WHERE id = :dup_id"""), {'canonical_id': canonical_id, 'dup_id': duplicate_id})s.commit()logger.info(f"Merged {duplicate_id} -> {canonical_id}")def pick_canonical(self, record_a: BusinessRecord, record_b: BusinessRecord) -> str:score_a = self._completeness_score(record_a)score_b = self._completeness_score(record_b)return record_a.id if score_a >= score_b else record_b.iddef run_bulk_dedup(self, batch_size: int = 500) -> dict:stats = {'checked': 0, 'definite_merged': 0, 'flagged': 0, 'errors': 0}for canonical_id, dup_id in self._find_phone_duplicates_sql():try: self.merge_into_canonical(canonical_id, dup_id)stats['definite_merged'] += 1except Exception as e: stats['errors'] += 1for canonical_id, dup_id in self._find_domain_duplicates_sql():try: self.merge_into_canonical(canonical_id, dup_id)stats['definite_merged'] += 1except Exception as e: stats['errors'] += 1return statsdef _check_phone(self, candidate: BusinessRecord) -> Optional[DuplicateMatch]:row = self.session.execute(text("""SELECT id FROM businessesWHERE phone_normalized = :phone AND id != :cidAND deleted_at IS NULL AND duplicate_status = 'original'LIMIT 1"""), {'phone': candidate.phone_normalized, 'cid': candidate.id}).fetchone()if row: return DuplicateMatch(existing_id=str(row[0]), candidate_id=candidate.id, match_type='phone_exact', confidence=1.0, is_definite=True)return Nonedef _check_domain(self, candidate: BusinessRecord) -> Optional[DuplicateMatch]:row = self.session.execute(text(""" SELECT id FROM businessesWHERE website_domain = :domain AND id != :cidAND deleted_at IS NULL AND duplicate_status = 'original'LIMIT 1"""), {'domain': candidate.website_domain, 'cid': candidate.id}).fetchone()if row:return DuplicateMatch(existing_id=str(row[0]), candidate_id=candidate.id,match_type='domain_exact', confidence=1.0, is_definite=True)return Nonedef _find_phone_duplicates_sql(self) -> list[tuple[str, str]]:rows = self.session.execute(text(""" SELECT MIN(id::text) AS canonical_id,UNNEST(ARRAY_REMOVE(ARRAY_AGG(id::textORDER BY created_at), MIN(id::text))) AS dup_idFROM businessesWHERE phone_normalized IS NOT NULL AND deleted_at IS NULL AND duplicate_status = 'original'GROUP BY phone_normalized HAVING COUNT(*) > 1""")).fetchall()return [(r[0], r[1]) for r in rows]def _find_domain_duplicates_sql(self) -> list[tuple[str, str]]:rows = self.session.execute(text("""SELECT MIN(id::text) AS canonical_id,UNNEST(ARRAY_REMOVE(ARRAY_AGG(id::textORDER BY
```

created_at), MIN(id::text))) AS dup_idFROM businessesWHERE website_domain IS NOT NULL AND deleted_at IS NULL AND duplicate_status = 'original'GROUP BY website_domain HAVING COUNT(*) > 1""")).fetchall()return [(r[0], r[1]) for r in rows]def _fetch_record(self, record_id: str, session: Session) -> Optional[dict]:row = session.execute(text("SELECT * FROM businesses WHERE id = :id"), {'id': record_id}).mappings().fetchone()return dict(row) if row else None @staticmethoddef _completeness_score(r: BusinessRecord) -> int:score = 0if r.email: score += 3if r.phone: score += 2if r.address_line1: score += 2if r.website_domain: score += 2if r.rating: score += 1if r.review_count: score += 1return score@staticmethoddef normalize_phone(raw: str) -> Optional[str]:if not raw: return Nonedigits = re.sub(r'\D', '', raw)if len(digits) == 11 and digits.startswith('1'): return digitsif len(digits) == 10: return '1' + digitsreturn None@staticmethoddef extract_domain(url: str) -> Optional[str]:if not url: return Noneif not url.startswith(('http://', 'https://')): url = 'https://' + urltry:parsed = urlparse(url)host = parsed.netloc.lower().lstrip('www.')return host or Noneexcept Exception:return None

3D. Status Tracking Schema

Table 7 — Status Enum State Transitions

Enum	States / Transitions
scrape_status_enum	pending rightarrow in_progress rightarrow completed / failed / skipped
email_status_enum	unknown rightarrow valid / invalid / catch_all / disposable / role_based / unverifiable / bounced
phone_status_enum	unknown rightarrow valid / invalid / disconnected / voip / mobile / landline
outreach_status_enum	not_contacted rightarrow queued rightarrow emailed / called rightarrow replied rightarrow interested / not_interested / unsubscribed /
job_status_enum	pending rightarrow claimed rightarrow running rightarrow completed / failed rightarrow dead (max retries exceeded)
duplicate_status_enum	original / duplicate / merged

Key Timestamp Fields on businesses

- last_scraped_at — updated every time the record is refreshed from source
- last_contacted_at — updated by outreach_log trigger or manual update
- next_follow_up_at — set by outreach workflow; indexed for scheduler queries
- created_at / updated_at — auto-managed by triggers
- deleted_at — soft delete; all active queries filter WHERE deleted_at IS NULL

```
from __future__ import annotationsimport csvimport loggingfrom dataclasses import dataclass, field
from datetime import datetimefrom pathlib import Pathfrom typing import Optionalfrom sqlalchemy
import textfrom sqlalchemy.orm import Sessionlogger = logging.getLogger(__name__)
ALL_COLUMNS = {'id': 'ID', 'name': 'Business Name', 'vertical': 'Vertical','sub_vertical':
'Sub-Vertical', 'address_line1': 'Address', 'city': 'City','state': 'State', 'zip_code': 'ZIP', 'county':
'County', 'phone': 'Phone','phone_status': 'Phone Status', 'website': 'Website', 'email': 'Email',
'email_status': 'Email Status', 'description': 'Description', 'rating': 'Rating','review_count': 'Review
Count', 'years_in_business': 'Years in Business','outreach_status': 'Outreach Status',
'last_contacted_at': 'Last Contacted','next_follow_up_at': 'Next Follow-Up', 'source_url': 'Source
URL','last_scraped_at': 'Last Scraped', 'created_at': 'Created At',}DEFAULT_COLUMNS= ['name',
'vertical', 'address_line1', 'city', 'state', 'zip_code','phone', 'phone_status', 'website', 'email',
'email_status','outreach_status', 'rating', 'review_count',]@dataclassclass ExportFilters:verticals:
list[str] = field(default_factory=list)states: list[str] = field(default_factory=list)zip_codes: list[str] =
field(default_factory=list)outreach_statuses: list[str] = field(default_factory=list)email_statuses:
list[str] = field(default_factory=list)phone_statuses: list[str] = field(default_factory=list)min_rating:
Optional[float] = Nonehas_email: Optional[bool] = Nonehas_phone: Optional[bool] = None
exclude_duplicates: bool = Truelimit: Optional[int] = None@dataclassclass ExportResult:filepath: str
total_records: intcolumns_exported: list[str]filters_applied: dictexport_duration_seconds: floatstats:
dictclass CSVExporter:def __init__(self, session: Session):self.session = sessiondef export(self,
output_path: str,columns: Optional[list[str]] = None,filters: Optional[ExportFilters] = None,
include_contacts: bool = False,) -> ExportResult:start = datetime.utcnow()columns = columns or
DEFAULT_COLUMNSfilters = filters or ExportFilters()invalid = [c for c in columns if c not in
ALL_COLUMNS]if invalid:raise ValueError(f"Unknown columns: {invalid}")query, params =
self._build_query(columns, filters)rows = self.session.execute(text(query), params).fetchall()
output_path = Path(output_path)output_path.parent.mkdir(parents=True,exist_ok=True)with
open(output_path, 'w', newline='', encoding='utf-8-sig') as f:writer = csv.writer(f,
quoting=csv.QUOTE_MINIMAL)writer.writerow([ALL_COLUMNS[c] for c in columns])for row in
rows:writer.writerow([self._format_value(v) for v in row])duration = (datetime.utcnow() -
start).total_seconds()stats = self._compute_stats(rows, columns)result = ExportResult(
filepath=str(output_path), total_records=len(rows),columns_exported=columns,
filters_applied=self._filters_to_dict(filters),export_duration_seconds=duration, stats=stats,)
self._print_summary(result)return resultdef _build_query(self, columns: list[str], filters:
ExportFilters) -> tuple[str, dict]:col_sql = ', '.join(f'b.{c}' for c in columns)where_clauses, params =
self._build_where_clauses(filters)where_sql = ('WHERE ' + ' AND '.join(where_clauses)) if
where_clauses else ''query = f"SELECT {col_sql} FROM businesses b {where_sql} ORDER BY
b.state, b.vertical, b.name"if filters.limit:query += f" LIMIT {filters.limit}"return query, paramsdef
_build_where_clauses(self, filters: ExportFilters, table_prefix: str = 'b') -> tuple[list[str], dict]:p =
table_prefixclauses = [f"{p}.deleted_at IS NULL"]params: dict = {}if filters.exclude_duplicates:
```

clauses.append(f"{p}.duplicate_status = 'original'")if filters.verticals:clauses.append(f"{p}.vertical = ANY(:verticals)")params['verticals'] = filters.verticalsif filters.states:clauses.append(f"{p}.state = ANY(:states)")params['states'] = [s.upper() for s in filters.states]if filters.email_statuses: clauses.append(f"{p}.email_status = ANY(:email_statuses)")params['email_statuses'] = filters.email_statusesif filters.min_rating is not None:clauses.append(f"{p}.rating >= :min_rating") params['min_rating'] = filters.min_ratingif filters.has_email is True:clauses.append(f"{p}.email IS NOT NULL AND {p}.email != ''")elif filters.has_email is False:clauses.append(f"({p}.email IS NULL OR {p}.email = '')")if filters.has_phone is True:clauses.append(f"{p}.phone IS NOT NULL AND {p}.phone != ''")return clauses, params@staticmethoddef _format_value(v) -> str:if v is None: return ''if isinstance(v, datetime): return v.strftime('%Y-%m-%d%H:%M:%S')if isinstance(v, list): return '; '.join(str(x) for x in v)return str(v)@staticmethoddef _compute_stats(rows, columns: list[str]) -> dict:if not rows: return {}from collections import Counterstats: dict = {'total': len(rows)} col_idx = {c: i for i, c in enumerate(columns)}if 'state' in col_idx:states = Counter(row[col_idx['state']] for row in rows if row[col_idx['state']])stats['by_state'] = dict(states.most_common(10))if 'vertical' in col_idx:verts = Counter(row[col_idx['vertical']] for row in rows if row[col_idx['vertical']])stats['by_vertical'] = dict(verts.most_common())if 'email_status' in col_idx:es = Counter(row[col_idx['email_status']] for row in rows)stats['email_status_breakdown'] = dict(es)return stats@staticmethoddef _filters_to_dict(f: ExportFilters) -> dict:return {'verticals': f.verticals, 'states': f.states,'email_statuses': f.email_statuses, 'has_email': f.has_email, 'has_phone': f.has_phone, 'exclude_duplicates': f.exclude_duplicates,'limit': f.limit,}@staticmethod def _print_summary(result: ExportResult) -> None:print(f"\n{'='*50}")print(f" EXPORT SUMMARY") print(f"{'='*50}")print(f" File: {result.filepath}")print(f" Records: {result.total_records:,}")print(f" Duration: {result.export_duration_seconds:.2f}s")if result.stats.get('by_vertical'):print(f" By Vertical:") for v, c in result.stats['by_vertical'].items():print(f" {v:<30} {c:>6,}")print(f"{'='*50}\n")

Section 4 — Build Sequence & Project Architecture

4A. Build Order — Fastest Path to 20,000 Contacts

Realistic throughput: 10 workers times ~0.3 results/sec/worker = ~3 results/sec = 10,800/hr = ~100,000/day. After 20% dedup rate: ~80,000 net new records/day. 20,000 contacts achievable in under 1 day of scraping.

Phase 1 — Days 1–2: Foundation

Day 1 morninggit init firstknock-scraper && cd firstknock-scraperpython -m venv venv && source venv/bin/activatepip install -r requirements.txtplaywright install chromium# Spin up DB docker-compose up -dsleep 5# Apply schemadocker exec -i firstknock_db psql -U firstknock firstknock < database/schema.sql# Download ZIP codescurl -L https://simplemaps.com/static/data/us-zips/1.82/basic/simplemaps_uszips_basicv1.82.zip-o uszips.zipunzip uszips.zip -d config/mv config/uszips.csv config/zip_codes.csv# Populate job queue

python main.py populate-jobs --verticals pest_control,solar,roofing --states TX,FL,CA# Verify queuepython main.py stats

Phase 2 — Days 3–5: Core Scraper

Start scraping with 5 workers (conservative start)python main.py scrape --workers 5 --source google_maps# Monitor in separate terminalpython main.py monitor# Open http://localhost:5001# Check statspython main.py stats# Scale up once stablepython main.py scrape --workers 10 --source google_maps

Phase 3 — Days 6–7: Email Discovery

Run email discovery on all businesses with websitespython main.py discover-emails --workers 5 # Check coveragepython main.py stats# Run dedup passpython main.py dedup

Phase 4 — Days 8–9: Scale & Reliability

- Stale job reaper active (resets jobs stuck >30min to pending)
- Proxy health check loop active
- DB connection pool configured (pool_size=20, max_overflow=10)
- Loguru rotating file handler active
- Dedup job scheduled (run every 6 hours)

Phase 5 — Day 10: Export & Handoff

Full exportpython main.py export \--output exports/firstknock_$(date +%Y%m%d).csv \ --has-email true \--email-status valid,catch_all \--exclude-duplicates# Export by vertical for targeted campaignspython main.py export --vertical pest_control --output exports/pest_control.csvpython main.py export --vertical solar --output exports/solar.csvpython main.py export --vertical roofing --output exports/roofing.csv

4B. Failure Points & Mitigations

1. Google Maps IP Ban

Symptom: CAPTCHA page, empty results, redirect to google.com/sorry

Rotate on every request — use residential proxies onlyREQUESTS_PER_PROXY_PER_HOUR = 120 # conservativeDELAY_BETWEEN_REQUESTS= random.uniform(2.5, 5.0)# CAPTCHA detectionif 'sorry' in page.url or await page.query_selector('#captcha-form'): proxy_manager.mark_banned(current_proxy)raise ProxyBannedException()

2. SMTP Port 25 Blocked by VPS

Table 8 — VPS Provider Port 25 Availability

VPS Provider	Port 25 Status	Recommendation
AWS EC2	Blocked by default	Requires support ticket — often denied
GCP Compute	Blocked	Requires justification form
DigitalOcean	Blocked on new accounts	Avoid for SMTP verification
Hetzner Cloud	OPEN by default	Best choice for SMTP verification
Vultr	Open	Good alternative
Linode/Akamai	Open	Good alternative

Port fallback codeSMTP_PORTS = [25, 587, 465]async def _try_connect(self, host: str) -> tuple[bool, int]:for port in SMTP_PORTS:try:reader, writer = await asyncio.wait_for(asyncio.open_connection(host, port), timeout=10)writer.close()return True, portexcept (ConnectionRefusedError, asyncio.TimeoutError, OSError):continuereturn False, 0

3. Catch-All Mail Servers

async def _detect_catch_all(self, domain: str, mx_host: str) -> bool:fake = f"zzz_catchall_test_{uuid.uuid4().hex[:8]}@{domain}"result = await self._smtp_check(fake, mx_host)return result.accepted # if True, it's a catch-all

Strategy: Mark email as catch_all (not invalid). Still include in exports — catch-all neq undeliverable. Lower confidence score (0.5 vs 0.95 for verified). Prioritize pattern-generated emails with highest confidence patterns (firstname.lastname@ > info@).

4. Duplicate Records at Scale

Run dedup after every 1,000 insertsif records_inserted % 1000 == 0: deduplicator.run_bulk_dedup()# Pre-insert check (fast path)existing = session.execute(text(""" SELECT id FROM businessesWHERE phone_normalized = :phone OR website_domain = :domain LIMIT 1"""), {'phone': phone_norm, 'domain': domain}).fetchone()if existing: update_existing(existing.id, new_data)else:insert_new(new_data)# Expected dedup rate: 15-25% across sources. Budget for it in record count targets.

5. Scraper Breaking on Google Maps UI Changes

Multi-selector fallback patternSELECTORS = {'result_items': ['div[role="feed"] > div', # current (2024-2025)'div.Nv2PK', # alternate'[data-result-index]', # fallback],'business_name': ['h1.DUwDvf', 'h1[data-attrid="title"]','.qrShPb span',],}def safe_select(page, selector_list: list[str]):for sel in selector_list:el = page.query_selector(sel)if el:return elreturn None# Monitoring: log selector hit rates# If a selector drops below 50% hit rate, alert and pause scraper# Pin Playwright version — do not auto-update

WRONG — loads everything into RAMall_businesses = session.query(Business).all()# RIGHT — stream in batchesdef stream_businesses(session, batch_size=500):offset = 0while True:batch = session.execute(text("SELECT * FROM businesses WHERE deleted_at IS NULL ""ORDER BY id LIMIT :limit OFFSET :offset"),{'limit': batch_size, 'offset': offset}).fetchall()if not batch:breakyield batchoffset += batch_sizesession.expire_all() # release SQLAlchemy identity map# Connection pool configengine = create_engine(settings.database_url(),pool_size=20,max_overflow=10, pool_pre_ping=True,pool_recycle=3600,)

7. Rate Limiting on SMTP Verification

from asyncio import Semaphorefrom collections import defaultdictdomain_semaphores: dict[str, Semaphore] = defaultdict(lambda: Semaphore(1))domain_last_request: dict[str, float] = {} DOMAIN_DELAY = 2.0async def verify_with_rate_limit(email: str) -> VerificationResult:domain = email.split('@')[1]async with domain_semaphores[domain]:last = domain_last_request.get(domain, 0)wait = DOMAIN_DELAY - (time.time() - last)if wait > 0:await asyncio.sleep(wait)result = await smtp_verifier.verify(email)domain_last_request[domain] = time.time()return result# Batch strategy: verify emails in domain-grouped batches# All @gmail.com together, all @yahoo.com together, etc.

8. Stale Data / Closed Businesses

CLOSED_INDICATORS = ['permanently closed', 'temporarily closed','closed', 'out of business',]if any(ind in listing_text.lower() for ind in CLOSED_INDICATORS):business.scrape_status = 'skipped' business.outreach_notes = 'Marked closed on source'continue# Periodic re-scrape: re-check records older than 90 daysstale_query = """UPDATE businesses SET scrape_status = 'pending' WHERE last_scraped_at < NOW() - INTERVAL '90 days'AND deleted_at IS NULL"""

4C. Complete Project Folder Structure

firstknock-scraper/%% %%README.md # Setup, usage, architecture overview% %requirements.txt # Pinned dependencies% %docker-compose.yml # PostgreSQL + pgAdmin% %.env.example # All env vars with descriptions% %.gitignore # venv/, .env, exports/, *.pyc, __pycache__/%% %config/% % %settings.py # Loads .env, exposes typed config object%% %verticals.py # Search query templates per vertical%% %zip_codes.csv # All US ZIP codes with city/state/county%% %scraper/% % %__init__.py%% %google_maps.py # Playwright scraper — main source%% %yelp.py # Yelp scraper (requests + BS4)%% %yellow_pages.py # YP scraper (requests + BS4)%% % % proxy_manager.py # Proxy pool, rotation, health checks%% %stealth.py # Playwright stealth config, UA rotation%% %discovery/%% %__init__.py%% %website_crawler.py # Crawl business websites for emails%% %smtp_verifier.py # Async SMTP email verification%% % % pattern_generator.py # Generate email patterns from name+domain%% %whois_lookup.py # WHOIS for registrant email%% %phone_normalizer.py # E.164 normalization via phonenumbers lib %% %database/%% %__init__.py%% %models.py # SQLAlchemy ORM models (mirrors schema.sql)

% % % schema.sql # Canonical schema — source of truth% % % deduplicator.py # Dedup engine (Section 3C)% % % exporter.py # CSV export (Section 3E)% % % queue/% % % __init__.py% % % job_manager.py # Job CRUD, claim/release, retry logic% % % worker.py # Worker pool, orchestrates scraper + DB write% % % monitor/% % % __init__.py% % % app.py # Flask dashboard (port 5001)% % % templates/% % % dashboard.html # Real-time stats dashboard% % % exports/ # .gitignored — CSV output files% % % logs/ # .gitignored — rotating log files% % % main.py # CLI entry point (argparse/click)# Data flow:# main.py# % % populate-jobs rightarrow job_manager.py rightarrow scrape_jobs table# % % scrape rightarrow worker.py# % % job_manager.py (claim job)# % % google_maps.py (scrape)# % % deduplicator.py (pre-insert check)# % % models.py (write to DB)# % % job_manager.py (complete job) # % % discover-emails rightarrow website_crawler.py + pattern_generator.py + smtp_verifier.py# % % export rightarrow exporter.py rightarrow exports/*.csv# % % monitor rightarrow app.py (Flask)

4D. Complete requirements.txt

requirements.txt — FirstKnock Scraper# Python 3.11+# % % Browser Automation

playwright==1.44.0

playwright-stealth==1.0.6# % % HTTP & Parsing

requests==2.32.3

aiohttp==3.9.5beautifulsoup4==4.12.3lxml==5.2.2httpx==0.27.0# % % DNS & Email

dnspython==2.6.1#

smtp verification is stdlib (smtplib) + dnspython# % % WHOIS

python-whois==0.9.4# % % Phone

phonenumbers==8.13.39# % % Fuzzy Matching

rapidfuzz==3.9.3# % %

Database

sqlalchemy==2.0.30psycopg2-binary==2.9.9alembic==1.13.1# % % Search (fallback discovery)

duckduckgo-search==6.2.4# % % User Agent

Rotation

fake-useragent==1.5.1#

% % Config

python-dotenv==1.0.1# % % Data Processing

pandas==2.2.2

numpy==1.26.4# % % Progress & Logging

tqdm==4.66.4loguru==0.7.2# % %

Web Dashboard

flask==3.0.3flask-cors==4.0.1# % % Async Utilities

asyncio-throttle==1.0.2

tenacity==8.3.0# % % Utilities

python-dateutil==2.9.0

```
pytz==2024.1 click==8.1.7rich==13.7.1# Key GitHub repos:# playwright-stealth:
https://github.com/AtuboDad/playwright_stealth# duckduckgo-search:
https://github.com/deedy5/duckduckgo_search# rapidfuzz: https://github.com/rapidfuzz/RapidFuzz
# loguru: https://github.com/Delgan/loguru# patchright:
https://github.com/Kaliiiiiiiiii-Vinyzu/patchright-python
```

Section 5 — Working Code: Complete Implementations

5A. config/settings.py

```
# config/settings.pyfrom __future__ import annotationsimport osfrom pathlib import Pathfrom
dotenv import load_dotenvBASE_DIR = Path(__file__).resolve().parent.parent
load_dotenv(BASE_DIR / '.env')class Settings:# % % Database
% % % % % % % % % % % % % % % % % % % % % % % % % % % % % % DB_HOST: str = % % % % % % % %
os.getenv('DB_HOST', 'localhost')DB_PORT: int = int(os.getenv('DB_PORT', '5432'))DB_NAME:
str = os.getenv('DB_NAME', 'firstknock')DB_USER: str = os.getenv('DB_USER', 'firstknock')
DB_PASSWORD: str = os.getenv('DB_PASSWORD', 'changeme')@classmethoddef
database_url(cls) -> str:return (f"postgresql+psycopg2://{cls.DB_USER}:{cls.DB_PASSWORD}"
f"@{cls.DB_HOST}:{cls.DB_PORT}/{cls.DB_NAME}")# % % Scraper
% % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %
SCRAPER_WORKERS: int = int(os.getenv('SCRAPER_WORKERS', '10'))
SCRAPER_DELAY_MIN: float = float(os.getenv('SCRAPER_DELAY_MIN', '2.0'))
SCRAPER_DELAY_MAX: float = float(os.getenv('SCRAPER_DELAY_MAX', '5.0'))
SCRAPER_HEADLESS: bool = os.getenv('SCRAPER_HEADLESS', 'true').lower() == 'true'
SCRAPER_TIMEOUT_MS: int = int(os.getenv('SCRAPER_TIMEOUT_MS', '30000'))
MAX_RESULTS_PER_QUERY: int = int(os.getenv('MAX_RESULTS_PER_QUERY', '60'))# % %
Proxies % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %
PROXY_LIST_FILE: str = os.getenv('PROXY_LIST_FILE', str(BASE_DIR / 'config' / 'proxies.txt'))
USE_PROXIES: bool = os.getenv('USE_PROXIES', 'false').lower() == 'true'# % % Email Verification
% % % % % % % % % % % % % % % % % % % % % % % % % % SMTP_TIMEOUT: int = % % % % % % % % % %
int(os.getenv('SMTP_TIMEOUT', '10'))SMTP_FROM_EMAIL: str =
os.getenv('SMTP_FROM_EMAIL', 'verify@firstknock.local')EMAIL_WORKERS: int =
int(os.getenv('EMAIL_WORKERS', '5'))DOMAIN_DELAY_SECS: float =
float(os.getenv('DOMAIN_DELAY_SECS', '2.0'))# % % Paths
% % % % % % % % % % % % % % % % % % % % % % % % % % % % % % EXPORTS_DIR: % % % % % % %
Path = BASE_DIR / 'exports'LOGS_DIR: Path = BASE_DIR / 'logs'ZIP_CSV: Path = BASE_DIR /
'config' / 'zip_codes.csv'# % % Monitor
% % % % % % % % % % % % % % % % % % % % % % % % % % % % % % MONITOR_PORT: int % % % % %
= int(os.getenv('MONITOR_PORT', '5001'))MONITOR_HOST: str = os.getenv('MONITOR_HOST',
'0.0.0.0')# % % Job Queue % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %
```

JOB_CLAIM_TIMEOUT_MINS: int = int(os.getenv('JOB_CLAIM_TIMEOUT_MINS', '30'))JOB_MAX_ATTEMPTS: int = int(os.getenv('JOB_MAX_ATTEMPTS','3'))JOB_RETRY_DELAY_SECS: int = int(os.getenv('JOB_RETRY_DELAY_SECS','300'))settings = Settings()

5B. config/verticals.py

config/verticals.pyfrom __future__ import annotationsVERTICALS: dict[str, dict] = {'pest_control': { 'display_name': 'Pest Control','search_queries': ['pest control company', 'exterminator', 'termite control','rodent control', 'bed bug treatment', 'mosquito control service',],'keywords': ['pest', 'exterminator', 'termite', 'rodent', 'bug', 'insect'],'naics_codes': ['561710'],},'solar': {'display_name': 'Solar Energy','search_queries': ['solar panel installation', 'solar energy company','residential solar installer', 'commercial solar contractor','solar roofing company',],'keywords': ['solar', 'photovoltaic', 'pv installation', 'renewable energy'],'naics_codes': ['238160', '221114'],},'home_security': { 'display_name': 'Home Security','search_queries': ['home security company', 'security alarm installation','home alarm system', 'security camera installation', 'smart home security',],'keywords': ['security', 'alarm', 'surveillance', 'monitoring'],'naics_codes': ['561621'],},'roofing': {'display_name': 'Roofing','search_queries': ['roofing contractor', 'roof repair company', 'roof replacement', 'commercial roofing', 'residential roofing company', 'storm damage roofing',],'keywords': ['roofing', 'roofer', 'roof repair', 'shingles'],'naics_codes': ['238160'],},'hvac': {'display_name': 'HVAC', 'search_queries': ['HVAC company', 'air conditioning repair', 'heating and cooling company','AC installation', 'furnace repair', 'heat pump installation',],'keywords': ['hvac', 'air conditioning', 'heating', 'cooling', 'furnace', 'ac repair'],'naics_codes': ['238220'],},'d2d_sales': {'display_name': 'D2D Sales', 'search_queries': ['door to door sales company', 'direct sales company','field sales company', 'canvassing company', 'direct marketing company',],'keywords': ['door to door', 'd2d', 'direct sales', 'field sales', 'canvassing'],'naics_codes': ['454390'],},}ALL_VERTICALS = list(VERTICALS.keys()) def get_search_queries(vertical: str) -> list[str]:return VERTICALS[vertical]['search_queries']def get_all_queries() -> list[tuple[str, str]]:pairs = []for vertical, config in VERTICALS.items():for query in config['search_queries']:pairs.append((vertical, query))return pairs

5C. .env.example

.env.example — copy to .env and fill in values# DatabaseDB_HOST=localhostDB_PORT=5432 DB_NAME=firstknockDB_USER=firstknockDB_PASSWORD=changeme_strong_password# pgAdminPGADMIN_EMAIL=admin@firstknock.localPGADMIN_PASSWORD=admin# Scraper SCRAPER_WORKERS=10SCRAPER_DELAY_MIN=2.0SCRAPER_DELAY_MAX=5.0 SCRAPER_HEADLESS=trueSCRAPER_TIMEOUT_MS=30000MAX_RESULTS_PER_QUERY=60 # Proxies (optional but recommended for Google Maps)USE_PROXIES=false PROXY_LIST_FILE=config/proxies.txt# Email VerificationSMTP_TIMEOUT=10 SMTP_FROM_EMAIL=verify@yourdomain.comEMAIL_WORKERS=5DOMAIN_DELAY_SECS=2.0 # MonitorMONITOR_PORT=5001MONITOR_HOST=0.0.0.0# Job Queue JOB_CLAIM_TIMEOUT_MINS=30JOB_MAX_ATTEMPTS=3JOB_RETRY_DELAY_SECS=300

```
#!/usr/bin/env python3# main.py — FirstKnock Scraper CLI entry pointfrom __future__ import annotationsimport asyncioimport sysfrom pathlib import Pathimport clickfrom loguru import logger from sqlalchemy import create_engine, textfrom sqlalchemy.orm import sessionmakerfrom config.settings import settingsfrom config.verticals import ALL_VERTICALS, get_all_queries settings.LOGS_DIR.mkdir(exist_ok=True)settings.EXPORTS_DIR.mkdir(exist_ok=True) logger.remove()logger.add(sys.stderr, level="INFO", colorize=True,format="{time:HH:mm:ss} | {level: <8} | {message}")logger.add(settings.LOGS_DIR / "firstknock_{time:YYYY-MM-DD}.log", rotation="100 MB", retention="30 days", level="DEBUG",)engine = create_engine( settings.database_url(),pool_size=20, max_overflow=10,pool_pre_ping=True, pool_recycle=3600,) SessionLocal = sessionmaker(bind=engine)@click.group()def cli():"""FirstKnock — Lead Generation Scraper"""pass@cli.command()@click.option('--verticals', default='all', help='Comma-separated verticals or "all"')@click.option('--states', default='all', help='Comma-separated state codes or "all"')@click.option('--priority', default=5, type=int, help='Job priority (1=high, 10=low)')def populate_jobs(verticals: str, states: str, priority: int): """Populate the scrape_jobs queue from ZIP codes and verticals."""import csvfrom queue.job_manager import JobManagersession = SessionLocal()jm = JobManager(session) zip_path = settings.ZIP_CSVif not zip_path.exists():logger.error(f"ZIP codes file not found: {zip_path}")sys.exit(1)with open(zip_path, newline='', encoding='utf-8') as f:reader = csv.DictReader(f)zip_rows = list(reader)logger.info(f"Loaded {len(zip_rows):,} ZIP codes")if states != 'all':state_list = [s.strip().upper() for s in states.split(',')]zip_rows = [r for r in zip_rows if r.get('state_id', '').upper() in state_list]vert_list = ALL_VERTICALS if verticals == 'all' else [v.strip() for v in verticals.split(',')]source_row = session.execute(text("SELECT id FROM sources WHERE name = 'google_maps'")).fetchone()source_id = source_row[0] if source_row else 1total_created = 0for vertical in vert_list:from config.verticals import get_search_queriesqueries = get_search_queries(vertical)for query in queries:state_cities: dict[str, list] = {}for row in zip_rows: state = row.get('state_id', '').upper()city = row.get('city', '')if state and city: state_cities.setdefault(state, []).append((city, row.get('zip', '')))for state, city_zips in state_cities.items():seen_cities = set()for city, zip_code in city_zips:if city in seen_cities:continue seen_cities.add(city)jm.create_job(source_id=source_id, vertical=vertical,search_query=f"{query} in {city}, {state}",zip_code=zip_code, state=state, city=city, priority=priority,)total_created += 1 session.commit()logger.success(f"Created {total_created:,} scrape jobs")session.close() @cli.command()@click.option('--workers', default=10, type=int)@click.option('--source', default='google_maps')@click.option('--limit', default=None, type=int)@click.option('--vertical', default=None)def scrape(workers: int, source: str, limit: int, vertical: str):"""Run the scraper with N concurrent workers."""from queue.worker import WorkerPoollogger.info(f"Starting scraper: {workers} workers, source={source}")pool = WorkerPool(engine=engine, num_workers=workers, source_name=source,job_limit=limit, vertical_filter=vertical)asyncio.run(pool.run())@cli.command() @click.option('--workers', default=5, type=int)@click.option('--limit', default=None, type=int)
```

```
@click.option('--vertical', default=None)@click.option('--state', default=None)def discover_emails(workers: int, limit: int, vertical: str, state: str):"""Run email discovery pipeline on businesses without emails."""from discovery.website_crawler import WebsiteCrawlerfrom discovery.smtp_verifier import SMTPVerifierfrom discovery.pattern_generator import PatternGeneratorasync def run():session = SessionLocal()where = ["b.email IS NULL", "b.website IS NOT NULL", "b.deleted_at IS NULL"]params: dict = {}if vertical:where.append("b.vertical = :vertical")params['vertical'] = verticalif state:where.append("b.state = :state")params['state'] = state.upper()query = f"""SELECT id, name, website, website_domain, verticalFROM businesses b WHERE {' AND '.join(where)}ORDER BY created_at DESC{"LIMIT :limit" if limit else ""}"""if limit: params['limit'] = limitbusinesses = session.execute(text(query), params).fetchall() logger.info(f"Discovering emails for {len(businesses):,} businesses")crawler = WebsiteCrawler() verifier = SMTPVerifier()generator = PatternGenerator()sem = asyncio.Semaphore(workers)async def process_one(biz):async with sem:try:emails = await crawler.find_emails(biz.website)if not emails and biz.website_domain:patterns = generator.generate(domain=biz.website_domain)emails = [p.email for p in patterns[:3]]best_email = Nonebest_status = 'unknown'for email in emails[:5]: result = await verifier.verify(email)if result.final_status == 'valid':best_email = emailbest_status = 'valid'breakelif result.final_status == 'catch_all' and not best_email:best_email = emailbest_status = 'catch_all'if best_email:session.execute(text("""UPDATE businessesSET email = :email, email_status = :statusWHERE id = :id"""), {'email': best_email, 'status': best_status, 'id': str(biz.id)}) session.commit()except Exception as e:logger.warning(f"Email discovery failed for {biz.id}: {e}") tasks = [process_one(b) for b in businesses]from tqdm.asyncio import tqdmawait tqdm.gather(*tasks, desc="Discovering emails")session.close()asyncio.run(run())@cli.command() @click.option('--output', required=True)@click.option('--vertical', default=None) @click.option('--state', default=None)@click.option('--email-status', default=None) @click.option('--outreach-status', default=None)@click.option('--has-email', default=None, type=bool)@click.option('--has-phone', default=None, type=bool)@click.option('--limit', default=None, type=int)@click.option('--exclude-duplicates/--include-duplicates', default=True)def export(output, vertical, state, email_status, outreach_status,has_email, has_phone, limit, exclude_duplicates):"""Export businesses to CSV."""from database.exporter import CSVExporter, ExportFilterssession = SessionLocal()exporter = CSVExporter(session)filters = ExportFilters( verticals=[vertical] if vertical else [],states=[s.strip() for s in state.split(',')] if state else [], email_statuses=[s.strip() for s in email_status.split(',')] if email_status else [], outreach_statuses=[s.strip() for s in outreach_status.split(',')] if outreach_status else [], has_email=has_email, has_phone=has_phone,limit=limit, exclude_duplicates=exclude_duplicates,) result = exporter.export(output_path=output, filters=filters)logger.success(f"Exported {result.total_records:,} records to {result.filepath}")session.close()@cli.command()def dedup(): """Run full deduplication pass on the database."""from database.deduplicator import Deduplicator session = SessionLocal()deduplicator = Deduplicator(session)logger.info("Running bulk deduplication...")stats = deduplicator.run_bulk_dedup()logger.success(f"Dedup complete: {stats}") session.close()@cli.command()def monitor():"""Start the Flask monitoring dashboard."""from
```

monitor.app import create_appapp = create_app(engine)logger.info(f"Dashboard at http://localhost:{settings.MONITOR_PORT}")app.run(host=settings.MONITOR_HOST, port=settings.MONITOR_PORT, debug=False)@cli.command()def stats():"""Print database statistics."""session = SessionLocal()rows = session.execute(text("""SELECT vertical, COUNT(*) AS total, COUNT(email) AS with_email,COUNT(phone) AS with_phone, ROUND(COUNT(email)::numeric / COUNT(*) * 100, 1) AS email_pctFROM businessesWHERE deleted_at IS NULL AND duplicate_status = 'original'GROUP BY vertical ORDER BY total DESC """)).fetchall()total = session.execute(text("SELECT COUNT(*) FROM businesses WHERE deleted_at IS NULL AND duplicate_status = 'original'")).scalar()print(f"\n{'='*65}")print(f" FIRSTKNOCK DATABASESTATS— {total:,} total records")print(f"{'='*65}")print(f" {'Vertical':<25} {'Total':>8}{'w/Email':>8} {'w/Phone':>8} {'Email%':>7}")print(f" {'-'*55}")for row in rows:print(f" {row[0]:<25} {row[1]:>8,} {row[2]:>8,} {row[3]:>8,} {row[4]:>6.1f}%")print(f"{'='*65}\n")session.close() if __name__ == '__main__':cli()

5E. queue/job_manager.py

queue/job_manager.pyfrom __future__ import annotationsimport socketimport uuidfrom datetime import datetime, timedeltafrom typing import Optionalfrom loguru import loggerfrom sqlalchemy import textfrom sqlalchemy.orm import Sessionfrom config.settings import settingsclass JobManager:def __init__(self, session: Session):self.session = sessionself.worker_id = f"{socket.gethostname()}-{uuid.uuid4().hex[:8]}"def create_job(self, source_id: int, vertical: str, search_query: str,zip_code=None, state=None, city=None, radius_miles=25, priority=5) -> int: result = self.session.execute(text("""INSERT INTO scrape_jobs(source_id, vertical, search_query, zip_code, state, city,radius_miles, priority, status)VALUES(:source_id, :vertical, :search_query, :zip_code, :state, :city,:radius_miles, :priority, 'pending')ON CONFLICT DO NOTHINGRETURNING id"""), {'source_id': source_id, 'vertical': vertical, 'search_query': search_query,'zip_code': zip_code, 'state': state, 'city': city,'radius_miles': radius_miles, 'priority': priority,})row = result.fetchone()return row[0] if row else Nonedef claim_next_job(self, source_name=None, vertical=None) -> Optional[dict]:"""Atomically claim the next available job using SELECT FOR UPDATE SKIP LOCKED."""extra_where = ""params: dict = {'worker_id': self.worker_id, 'now': datetime.utcnow()}if source_name:extra_where += " AND s.name = :source_name"params['source_name'] = source_nameif vertical:extra_where += " AND j.vertical = :vertical"params['vertical'] = verticalrow = self.session.execute(text(f"""WITH next_job AS (SELECT j.idFROM scrape_jobs jJOIN sources s ON s.id = j.source_idWHERE j.status = 'pending'AND (j.next_retry_at IS NULL OR j.next_retry_at <= :now){extra_where}ORDER BY j.priority ASC, j.id ASCLIMIT 1FOR UPDATE OF j SKIP LOCKED)UPDATE scrape_jobsSET status = 'claimed', worker_id = :worker_id,claimed_at = :now, attempts = attempts + 1FROM next_jobWHERE scrape_jobs.id = next_job.idRETURNING scrape_jobs.*"""), params).mappings().fetchone()if row:self.session.commit()return dict(row)return Nonedef start_job(self, job_id: int) -> None:self.session.execute(text("""UPDATE scrape_jobs SET status = 'running', started_at = NOW() WHERE id = :id"""), {'id': job_id})self.session.commit()def

complete_job(self, job_id: int, results_found: int, results_saved: int) -> None:self.session.execute(text("""UPDATE scrape_jobsSET status = 'completed', completed_at = NOW(),results_found = :found, results_saved = :saved, worker_id = NULLWHERE id = :id"""), {'id': job_id, 'found': results_found, 'saved': results_saved})self.session.commit()def fail_job(self, job_id: int, error: str, traceback=None) -> None:retry_at = datetime.utcnow() + timedelta(seconds=settings.JOB_RETRY_DELAY_SECS)self.session.execute(text("""UPDATE scrape_jobsSET status = CASEWHEN attempts >= max_attempts THEN 'dead'ELSE 'pending' END,next_retry_at = CASEWHEN attempts >= max_attempts THEN NULLELSE :retry_atEND, error_message = :error,error_traceback = :tb,worker_id = NULLWHERE id = :id"""), {'id': job_id, 'error': error[:500], 'tb': traceback, 'retry_at': retry_at})self.session.commit()def reap_stale_claims(self) -> int:cutoff = datetime.utcnow() - timedelta(minutes=settings.JOB_CLAIM_TIMEOUT_MINS)result = self.session.execute(text(""" UPDATE scrape_jobsSET status = 'pending', worker_id = NULL, claimed_at = NULL, started_at = NULLWHERE status IN ('claimed', 'running') AND claimed_at < :cutoffRETURNING id"""), {'cutoff': cutoff})count = len(result.fetchall())if count:self.session.commit()logger.warning(f"Reaped {count} stale jobs")return countdef get_queue_stats(self) -> dict:rows = self.session.execute(text("SELECT status, COUNT(*) as cnt FROM scrape_jobs GROUP BY status")).fetchall()return {r[0]: r[1] for r in rows}def get_progress(self) -> dict:row = self.session.execute(text("""SELECTCOUNT(*) FILTER (WHERE status = 'completed') AS completed,COUNT(*) FILTER (WHERE status = 'pending') AS pending,COUNT(*) FILTER (WHERE status = 'running') AS running,COUNT(*) FILTER (WHERE status = 'failed') AS failed,COUNT(*) FILTER (WHERE status = 'dead') AS dead,COUNT(*) AS total,SUM(results_saved) FILTER (WHERE status = 'completed') AS total_savedFROM scrape_jobs""")).mappings().fetchone()return dict(row) if row else {}

5F. queue/worker.py

queue/worker.pyfrom __future__ import annotationsimport asyncioimport tracebackfrom typing import Optionalfrom loguru import loggerfrom sqlalchemy import Enginefrom sqlalchemy.orm import sessionmakerfrom config.settings import settingsfrom queue.job_manager import JobManagerfrom scraper.google_maps import GoogleMapsScraperclass Worker:def __init__(self, worker_id: int, engine: Engine, source_name: str):self.worker_id = worker_idself.engine = engine self.source_name = source_nameself.SessionLocal = sessionmaker(bind=engine) self.jobs_processed = 0self.records_saved = 0async def run(self, job_limit=None, vertical_filter=None, stop_event=None) -> None:session = self.SessionLocal()jm = JobManager(session)scraper = GoogleMapsScraper(session)try:await scraper.init_browser()while True:if stop_event and stop_event.is_set():breakif job_limit and self.jobs_processed >= job_limit: breakjob = jm.claim_next_job(source_name=self.source_name, vertical=vertical_filter)if not job: logger.debug(f"Worker {self.worker_id}: queue empty, sleeping 10s")await asyncio.sleep(10) continuejm.start_job(job['id'])logger.info(f"Worker {self.worker_id} | Job {job['id']} | {job['vertical']} | {job['search_query']}")try:results = await scraper.scrape_query(query=job['search_query'],

vertical=job['vertical'], max_results=settings.MAX_RESULTS_PER_QUERY)saved = scraper.save_results(results, source_id=job['source_id'])jm.complete_job(job['id'], results_found=len(results), results_saved=saved)self.jobs_processed += 1self.records_saved += savedlogger.success(f"Worker {self.worker_id} | Job {job['id']} done | {len(results)} found, {saved} saved")except Exception as e:tb = traceback.format_exc()logger.error(f"Worker {self.worker_id} | Job {job['id']} failed: {e}")jm.fail_job(job['id'], error=str(e), traceback=tb)import randomawait asyncio.sleep(random.uniform(settings.SCRAPER_DELAY_MIN, settings.SCRAPER_DELAY_MAX))finally:await scraper.close_browser()session.close() logger.info(f"Worker {self.worker_id} finished: {self.jobs_processed} jobs, {self.records_saved} records")class WorkerPool:def __init__(self, engine: Engine, num_workers: int, source_name='google_maps',job_limit=None, vertical_filter=None):self.engine = engine self.num_workers = num_workersself.source_name = source_nameself.job_limit = job_limit self.vertical_filter = vertical_filterasync def run(self) -> None:stop_event = asyncio.Event()workers = [Worker(i, self.engine, self.source_name) for i in range(self.num_workers)]async def reaper(): SessionLocal = sessionmaker(bind=self.engine)while not stop_event.is_set():await asyncio.sleep(300)session = SessionLocal()jm = JobManager(session)jm.reap_stale_claims() session.close()tasks = [asyncio.create_task(w.run(job_limit=self.job_limit, vertical_filter=self.vertical_filter,stop_event=stop_event,))for w in workers] tasks.append(asyncio.create_task(reaper()))try:await asyncio.gather(*tasks)except KeyboardInterrupt:logger.info("Shutting down workers...")stop_event.set()await asyncio.gather(*tasks, return_exceptions=True)total_saved = sum(w.records_saved for w in workers)total_jobs = sum(w.jobs_processed for w in workers)logger.success(f"Pool complete: {total_jobs} jobs, {total_saved} records saved")

5G. scraper/google_maps.py

scraper/google_maps.pyfrom __future__ import annotationsimport asyncioimport jsonimport randomimport reimport tracebackfrom dataclasses import dataclass, fieldfrom typing import Optionalfrom urllib.parse import urlparse, quote_plusfrom loguru import loggerfrom playwright.async_api import (async_playwright, Browser, BrowserContext, Page,TimeoutError as PlaywrightTimeout,)from sqlalchemy import textfrom sqlalchemy.orm import Sessionfrom config.settings import settingsfrom database.deduplicator import Deduplicatorfrom scraper.stealth import apply_stealthfrom scraper.proxy_manager import ProxyManagerSEL = {'result_feed': ['div[role="feed"]','div.m6QErb[aria-label]','#QA0Szd div[role="feed"]',],'result_item': ['div[role="feed"] > div[jsaction]','div.Nv2PK','a.hfpxzc',],'business_name': ['h1.DUwDvf','h1[data-attrid="title"]', '.qrShPb span','h1',],'address': ['button[data-item-id="address"] .Io6YTe','[data-tooltip="Copy address"] .Io6YTe','div.rogA2c .Io6YTe',],'phone': ['button[data-item-id^="phone:tel:"] .Io6YTe', '[data-tooltip="Copy phone number"] .Io6YTe','span[aria-label^="Phone:"]',],'website': ['a[data-item-id="authority"]','a[aria-label^="Website:"]','a[href^="http"][data-tooltip="Open website"]',],'rating': ['div.F7nice span[aria-hidden="true"]','span.ceNzKf','div.fontDisplayLarge',],'review_count':

['div.F7nice span[aria-label$="reviews"]','button[aria-label$="reviews"] span',],'category': ['button.DkEaL','span.YkuOqf','.LrzXr',],'closed_indicator': ['span.ZDu9vd', 'span[aria-label*="Closed"]','span[aria-label*="Permanently closed"]',],}CLOSED_TEXTS = {'permanently closed', 'temporarily closed', 'closed'}@dataclassclass ScrapedBusiness:name: str vertical: straddress_raw: Optional[str] = Noneaddress_line1: Optional[str] = Nonecity: Optional[str] = Nonestate: Optional[str] = Nonezip_code: Optional[str] = Nonephone: Optional[str] = None website: Optional[str] = Nonewebsite_domain: Optional[str] = Nonerating: Optional[float] = None review_count: Optional[int] = Nonecategories: list[str] = field(default_factory=list)description: Optional[str] = Nonehours_json: Optional[dict] = Nonesource_url: Optional[str] = None source_listing_id: Optional[str] = Noneis_closed: bool = Falseraw_data: dict = field(default_factory=dict)class GoogleMapsScraper:def __init__(self, session: Session): self.session = sessionself.deduplicator = Deduplicator(session)self.proxy_manager = ProxyManager() if settings.USE_PROXIES else Noneself._playwright = Noneself._browser: Optional[Browser] = Noneself._context: Optional[BrowserContext] = Noneasync def init_browser(self) -> None:self._playwright = await async_playwright().start()launch_kwargs = { 'headless': settings.SCRAPER_HEADLESS,'args': ['--no-sandbox', '--disable-setuid-sandbox', '--disable-blink-features=AutomationControlled','--disable-infobars', '--window-size=1366,768', '--disable-dev-shm-usage', '--disable-gpu',],}if self.proxy_manager:proxy = self.proxy_manager.get_proxy()if proxy:launch_kwargs['proxy'] = proxyself._browser = await self._playwright.chromium.launch(**launch_kwargs)self._context = await self._browser.new_context(viewport={'width': 1366, 'height': 768},user_agent=self._random_ua(), locale='en-US',timezone_id='America/New_York'geolocation={'latitude': 40.7128, 'longitude': -74.0060},permissions=['geolocation'],)await apply_stealth(self._context)await self._context.route('**/*.{png,jpg,jpeg,gif,svg,ico,woff,woff2,ttf,mp4,webm}',lambda route: route.abort(),)async def close_browser(self) -> None:if self._context: await self._context.close()if self._browser: await self._browser.close()if self._playwright: await self._playwright.stop()async def scrape_query(self, query: str, vertical: str, max_results: int = 60) -> list[ScrapedBusiness]:page = await self._context.new_page()results: list[ScrapedBusiness] = []try:url = f"https://www.google.com/maps/search/{quote_plus(query)}"await page.goto(url, wait_until='domcontentloaded', timeout=settings.SCRAPER_TIMEOUT_MS)await self._random_delay(1.5, 3.0)if await self._is_captcha(page):logger.warning("CAPTCHA detected") raise CaptchaException("Google Maps CAPTCHA")feed = await self._wait_for_feed(page)if not feed:return resultsawait self._scroll_results(page, feed, target_count=max_results)items = await self._get_result_items(page)for i, item in enumerate(items[:max_results]):try:biz = await self._extract_listing(page, item, vertical, i)if biz and not biz.is_closed:results.append(biz)await self._random_delay(0.5, 1.5)except PlaywrightTimeout:logger.debug(f"Timeout on item {i}, skipping")except Exception as e:logger.debug(f"Error on item {i}: {e}")except CaptchaException: raiseexcept Exception as e:logger.error(f"scrape_query failed for '{query}': {e}")finally:await page.close()return resultsasync def _extract_listing(self, page: Page, item, vertical: str, index: int) -> Optional[ScrapedBusiness]:try:await item.click()await self._random_delay(1.5, 2.5)await

```
page.wait_for_selector(', '.join(SEL['business_name']), timeout=8000)except PlaywrightTimeout:return Nonebiz = ScrapedBusiness(name='', vertical=vertical)biz.source_url = page.urlcid_match = re.search(r'!1s(0x[0-9a-f]+:[0-9a-fx]+)', page.url)if cid_match:biz.source_listing_id = cid_match.group(1)name_el = await self._safe_query(page, SEL['business_name'])if name_el: biz.name = (await name_el.inner_text()).strip()if not biz.name:return Noneclosed_el = await self._safe_query(page, SEL['closed_indicator'])if closed_el:closed_text = (await closed_el.inner_text()).strip().lower()if any(c in closed_text for c in CLOSED_TEXTS):biz.is_closed = Truereturn bizaddr_el = await self._safe_query(page, SEL['address'])if addr_el:biz.address_raw = (await addr_el.inner_text()).strip()self._parse_address(biz)phone_el = await self._safe_query(page, SEL['phone'])if phone_el:biz.phone = (await phone_el.inner_text()).strip()website_el = await self._safe_query(page, SEL['website'])if website_el:href = await website_el.get_attribute('href')if href:biz.website = self._clean_website_url(href)biz.website_domain = Deduplicator.extract_domain(biz.website)rating_el = await self._safe_query(page, SEL['rating'])if rating_el:try:biz.rating = float((await rating_el.inner_text()).strip().replace(',', '.'))except ValueError: passreview_el = await self._safe_query(page, SEL['review_count'])if review_el:review_text = (await review_el.inner_text()).strip()nums = re.findall(r'[\d,]+', review_text)if nums:try:biz.review_count = int(nums[0].replace(',', ''))except ValueError:passcat_els = await page.query_selector_all(SEL['category'][0])biz.categories = []for el in cat_els[:5]:t = (await el.inner_text()).strip()if t: biz.categories.append(t)biz.raw_data = {'name': biz.name, 'address_raw': biz.address_raw, 'phone': biz.phone,'website': biz.website, 'rating': biz.rating, 'review_count': biz.review_count,'categories': biz.categories, 'source_url': biz.source_url,}return bizdef save_results(self, results: list[ScrapedBusiness], source_id: int) -> int:saved = 0for biz in results:try: saved += self._upsert_business(biz, source_id)except Exception as e:logger.error(f"Failed to save {biz.name}: {e}")self.session.commit()return saveddef _upsert_business(self, biz: ScrapedBusiness, source_id: int) -> int:phone_norm = Deduplicator.normalize_phone(biz.phone or '')existing_id = Noneif phone_norm:row = self.session.execute(text("SELECT id FROM businesses WHERE phone_normalized = :p AND deleted_at IS NULL LIMIT 1"), {'p': phone_norm}).fetchone() if row: existing_id = str(row[0])if not existing_id and biz.website_domain:row = self.session.execute(text("SELECT id FROM businesses WHERE website_domain = :d AND deleted_at IS NULL LIMIT 1"), {'d': biz.website_domain}).fetchone()if row: existing_id = str(row[0])if existing_id:self.session.execute(text("""UPDATE businesses SET last_scraped_at = NOW(), scrape_status = 'completed',all_source_ids = array_append(all_source_ids, :source_id::integer), rating = COALESCE(rating, :rating),review_count = COALESCE(review_count, :review_count), raw_data = :raw_data::jsonbWHERE id = :id"""), {'id': existing_id, 'source_id': source_id, 'rating': biz.rating,'review_count': biz.review_count, 'raw_data': json.dumps(biz.raw_data)})return 0 self.session.execute(text("""INSERT INTO businesses (name, vertical, address_line1, city, state, zip_code,phone, phone_normalized, website, website_domain,rating, review_count, categories, source_id, source_listing_id, source_url,all_source_ids, scrape_status, last_scraped_at, raw_data) VALUES (:name, :vertical, :address_line1, :city, :state, :zip_code,:phone, :phone_norm, :website, :website_domain,:rating, :review_count, :categories,:source_id, :source_listing_id, :source_url,
```

ARRAY[:source_id_arr::integer], 'completed', NOW(), :raw_data::jsonb)"""), {'name': biz.name, 'vertical': biz.vertical, 'address_line1': biz.address_line1,'city': biz.city, 'state': biz.state, 'zip_code': biz.zip_code,'phone': biz.phone, 'phone_norm': phone_norm, 'website': biz.website, 'website_domain': biz.website_domain, 'rating': biz.rating,'review_count': biz.review_count, 'categories': biz.categories or [],'source_id': source_id, 'source_listing_id': biz.source_listing_id, 'source_url': biz.source_url, 'source_id_arr': source_id,'raw_data': json.dumps(biz.raw_data),}) return 1async def _wait_for_feed(self, page: Page):for sel in SEL['result_feed']:try:el = await page.wait_for_selector(sel, timeout=10000)if el: return elexcept PlaywrightTimeout:continuereturn Noneasync def _get_result_items(self, page: Page) -> list:for sel in SEL['result_item']:items = await page.query_selector_all(sel)if items: return itemsreturn []async def _scroll_results(self, page: Page, feed, target_count: int) -> None:prev_count = 0stall_count = 0for _ in range(20):items = await self._get_result_items(page)current_count = len(items)if current_count >= target_count: breakif current_count == prev_count:stall_count += 1if stall_count >= 3: breakelse:stall_count = 0 prev_count = current_countawait feed.evaluate('el => el.scrollBy(0, 800)')await self._random_delay(1.0, 2.0)async def _safe_query(self, page: Page, selectors: list[str]):for sel in selectors:try:el = await page.query_selector(sel)if el: return elexcept Exception:continuereturn None async def _is_captcha(self, page: Page) -> bool:url = page.urlif 'google.com/sorry' in url or 'recaptcha' in url: return Truecaptcha_sel = await page.query_selector('#captcha-form, .g-recaptcha, iframe[src*="recaptcha"]')return captcha_sel is not None@staticmethoddef _parse_address(biz: ScrapedBusiness) -> None:if not biz.address_raw: returnpattern = r'^(.+),\s*([^,]+),\s*([A-Z]{2}) (\d{5}(?:-\d{4})?)$' m = re.match(pattern, biz.address_raw.strip())if m: biz.address_line1 = m.group(1).strip()biz.city = m.group(2).strip()biz.state = m.group(3).strip() biz.zip_code = m.group(4).strip()else:state_zip = re.search(r'\b([A-Z]{2}) (\d{5})\b', biz.address_raw)if state_zip:biz.state = state_zip.group(1)biz.zip_code = state_zip.group(2) biz.address_line1 = biz.address_raw@staticmethoddef _clean_website_url(href: str) -> str:if '/url?q=' in href:match = re.search(r'/url\?q=([^&]+)', href)if match:from urllib.parse import unquote return unquote(match.group(1))return href@staticmethoddef _random_ua() -> str:agents = ['Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/124.0.0.0 Safari/537.36','Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/124.0.0.0 Safari/537.36','Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/123.0.0.0 Safari/537.36',] return random.choice(agents)@staticmethodasync def _random_delay(min_s: float, max_s: float) -> None:await asyncio.sleep(random.uniform(min_s, max_s))class CaptchaException(Exception): pass

5H. scraper/stealth.py

scraper/stealth.pyfrom __future__ import annotationsfrom playwright.async_api import BrowserContextasync def apply_stealth(context: BrowserContext) -> None:stealth_js = """ Object.defineProperty(navigator, 'webdriver', { get: () => undefined });

Object.defineProperty(navigator, 'plugins', {get: () => [{ name: 'Chrome PDF Plugin', filename: 'internal-pdf-viewer' },{ name: 'Chrome PDF Viewer', filename: 'mhjfbmdgcfjbbpaeojofohoefgiehjai' },{ name: 'Native Client', filename: 'internal-nacl-plugin' },],});Object.defineProperty(navigator, 'languages', { get: () => ['en-US', 'en'] });const originalQuery = window.navigator.permissions.query;window.navigator.permissions.query = (parameters) => (parameters.name === 'notifications'? Promise.resolve({ state: Notification.permission }): originalQuery(parameters));window.chrome = { runtime: {},loadTimes: function() {},csi: function() {},app: {},};Object.defineProperty(HTMLIFrameElement.prototype, 'contentWindow', {get: function() { return window; },});Object.defineProperty(screen, 'width', { get: () => 1366 });Object.defineProperty(screen, 'height', { get: () => 768 });Object.defineProperty(screen, 'availWidth', { get: () => 1366 });Object.defineProperty(screen, 'availHeight', { get: () => 728 });Object.defineProperty(screen, 'colorDepth', { get: () => 24 });Object.defineProperty(screen, 'pixelDepth', { get: () => 24 });"""await context.add_init_script(stealth_js)

5l. scraper/proxy_manager.py

scraper/proxy_manager.pyfrom __future__ import annotationsimport randomimport timefrom dataclasses import dataclassfrom pathlib import Pathfrom typing import Optionalfrom loguru import loggerfrom config.settings import settings@dataclassclass Proxy:server: strusername: Optional[str] = Nonepassword: Optional[str] = Nonebanned_until: float = 0.0request_count: int = 0last_used: float = 0.0@propertydef is_available(self) -> bool:return time.time() > self.banned_untildef to_dict(self) -> dict:d = {'server': self.server}if self.username: d['username'] = self.usernameif self.password: d['password'] = self.passwordreturn dclass ProxyManager:def __init__(self): self.proxies: list[Proxy] = []self.current_proxy: Optional[Proxy] = Noneself._load_proxies()def _load_proxies(self) -> None:proxy_file = Path(settings.PROXY_LIST_FILE)if not proxy_file.exists(): logger.warning(f"Proxy file not found: {proxy_file}")returnwith open(proxy_file) as f:for line in f:line = line.strip()if not line or line.startswith('#'): continueproxy = self._parse_proxy_line(line)if proxy: self.proxies.append(proxy)logger.info(f"Loaded {len(self.proxies)} proxies")@staticmethoddef _parse_proxy_line(line: str) -> Optional[Proxy]:try:from urllib.parse import urlparseparsed = urlparse(line)server = f"{parsed.scheme}://{parsed.hostname}:{parsed.port}"return Proxy(server=server, username=parsed.username, password=parsed.password)except Exception: return Nonedef get_proxy(self) -> Optional[dict]:if not self.proxies: return Noneavailable = [p for p in self.proxies if p.is_available]if not available:logger.warning("All proxies banned — waiting 60s") time.sleep(60)for p in self.proxies: p.banned_until = 0available = self.proxiesproxy = min(available, key=lambda p: p.last_used)proxy.last_used = time.time()proxy.request_count += 1 self.current_proxy = proxyreturn proxy.to_dict()def mark_banned(self, proxy: Optional[Proxy], ban_duration_secs: int = 3600) -> None:if proxy:proxy.banned_until = time.time() + ban_duration_secslogger.warning(f"Proxy banned for {ban_duration_secs}s: {proxy.server}")def get_stats(self) -> dict:return {'total': len(self.proxies),'available': sum(1 for p in self.proxies if p.is_available),'banned': sum(1 for p in self.proxies if not p.is_available),}

```
# discovery/smtp_verifier.pyfrom __future__ import annotationsimport asyncioimport reimport smtplibimport socketimport uuidfrom dataclasses import dataclassfrom typing import Optional import dns.resolverfrom loguru import loggerfrom config.settings import settingsROLE_PREFIXES = {'info', 'admin', 'contact', 'support', 'sales', 'hello', 'help','noreply', 'no-reply', 'webmaster', 'postmaster', 'abuse','billing', 'marketing', 'team', 'office', 'mail',}DISPOSABLE_DOMAINS = { 'mailinator.com', 'guerrillamail.com', 'tempmail.com', 'throwaway.email','yopmail.com', '10minutemail.com', 'trashmail.com', 'sharklasers.com',}@dataclassclass VerificationResult:email: strsyntax_valid: bool = Falsemx_found: bool = Falsemx_records: list = Nonesmtp_connectable: bool = Falsesmtp_accepted: bool = Falseis_catch_all: bool = Falseis_disposable: bool = False is_role_based: bool = Falsefinal_status: str = 'unknown'confidence: float = 0.0smtp_response: Optional[str] = Nonesmtp_code: Optional[int] = Nonedef __post_init__(self):if self.mx_records is None:self.mx_records = []class SMTPVerifier:def __init__(self):self._domain_semaphores: dict[str, asyncio.Semaphore] = {}self._domain_last_request: dict[str, float] = {}self._mx_cache: dict[str, list[str]] = {}self._catch_all_cache: dict[str, bool] = {}async def verify(self, email: str) -> VerificationResult:result = VerificationResult(email=email)if not self._check_syntax(email): result.final_status = 'invalid'return resultresult.syntax_valid = Truedomain = email.split('@')[1].lower()if domain in DISPOSABLE_DOMAINS:result.is_disposable = True result.final_status = 'disposable'return resultlocal = email.split('@')[0].lower()if local in ROLE_PREFIXES:result.is_role_based = Truemx_hosts = await self._get_mx_records(domain)if not mx_hosts:result.final_status = 'invalid'return resultresult.mx_found = Trueresult.mx_records = mx_hostsawait self._rate_limit(domain)smtp_result = await asyncio.get_event_loop().run_in_executor(None, self._smtp_check, email, mx_hosts[0]) result.smtp_connectable = smtp_result['connectable']result.smtp_accepted = smtp_result['accepted']result.smtp_response = smtp_result.get('response')result.smtp_code = smtp_result.get('code')if not result.smtp_connectable:result.final_status = 'unverifiable' result.confidence = 0.4return resultif result.smtp_accepted:result.is_catch_all = await self._detect_catch_all(domain, mx_hosts[0])result.final_status, result.confidence = self._compute_status(result)return resultasync def verify_batch(self, emails: list[str], concurrency: int = 10) -> list[VerificationResult]:sem = asyncio.Semaphore(concurrency)async def bounded_verify(email: str) -> VerificationResult:async with sem:return await self.verify(email)return await asyncio.gather(*[bounded_verify(e) for e in emails])@staticmethoddef _check_syntax(email: str) -> bool:pattern = r'^[a-zA-Z0-9._%+\-]+@[a-zA-Z0-9.\-]+\.[a-zA-Z]{2,}$'return bool(re.match(pattern, email)) and len(email) <= 254async def _get_mx_records(self, domain: str) -> list[str]:if domain in self._mx_cache:return self._mx_cache[domain]try:records = await asyncio.get_event_loop().run_in_executor(None, self._resolve_mx, domain) self._mx_cache[domain] = recordsreturn recordsexcept Exception:self._mx_cache[domain] = [] return []@staticmethoddef _resolve_mx(domain: str) -> list[str]:try:answers = dns.resolver.resolve(domain, 'MX', lifetime=10)return [str(r.exchange).rstrip('.') for r in
```

sorted(answers, key=lambda x: x.preference)]except Exception:return []def _smtp_check(self, email: str, mx_host: str) -> dict:result = {'connectable': False, 'accepted': False, 'response': None, 'code': None}for port in [25, 587]:try:with smtplib.SMTP(timeout=settings.SMTP_TIMEOUT) as smtp: smtp.connect(mx_host, port)result['connectable'] = Truesmtp.ehlo_or_helo_if_needed() smtp.mail(settings.SMTP_FROM_EMAIL)code, msg = smtp.rcpt(email)result['code'] = code result['response'] = msg.decode('utf-8', errors='replace') if isinstance(msg, bytes) else str(msg) result['accepted'] = code in (250, 251)return resultexcept smtplib.SMTPConnectError:continue except smtplib.SMTPRecipientsRefused as e:result['connectable'] = Truefor addr, (code, msg) in e.recipients.items():result['code'] = coderesult['response'] = msg.decode('utf-8', errors='replace') if isinstance(msg, bytes) else str(msg)result['accepted'] = Falsereturn resultexcept (socket.timeout, ConnectionRefusedError, OSError):continuereturn resultasync def _detect_catch_all(self, domain: str, mx_host: str) -> bool:if domain in self._catch_all_cache:return self._catch_all_cache[domain] fake_email = f"zzz_test_{uuid.uuid4().hex[:8]}@{domain}"result = await asyncio.get_event_loop().run_in_executor(None, self._smtp_check, fake_email, mx_host) is_catch_all = result['accepted']self._catch_all_cache[domain] = is_catch_allreturn is_catch_all async def _rate_limit(self, domain: str) -> None:if domain not in self._domain_semaphores: self._domain_semaphores[domain] = asyncio.Semaphore(1)async with self._domain_semaphores[domain]:import timelast = self._domain_last_request.get(domain, 0.0) wait = settings.DOMAIN_DELAY_SECS - (time.time() - last)if wait > 0:await asyncio.sleep(wait) self._domain_last_request[domain] = time.time()@staticmethoddef _compute_status(r: VerificationResult) -> tuple[str, float]:if r.is_disposable: return 'disposable', 0.0if not r.mx_found: return 'invalid', 0.0if not r.smtp_connectable: return 'unverifiable', 0.35if r.smtp_accepted and r.is_catch_all: return 'catch_all', 0.5if r.smtp_accepted and not r.is_catch_all:confidence = 0.85 if r.is_role_based else 0.95return 'valid', confidenceif not r.smtp_accepted: return 'invalid', 0.05return 'unknown', 0.0

5K. discovery/pattern_generator.py

discovery/pattern_generator.pyfrom __future__ import annotationsimport refrom dataclasses import dataclassfrom typing import Optional@dataclassclass EmailPattern:email: strpattern_name: strconfidence: float# 14 patterns ordered by typical prevalencePATTERN_TEMPLATES= [('info', lambda f, l, d: f"info@{d}", 0.70),('contact', lambda f, l, d: f"contact@{d}", 0.55),('hello', lambda f, l, d: f"hello@{d}", 0.45),('admin', lambda f, l, d: f"admin@{d}", 0.40),('office', lambda f, l, d: f"office@{d}", 0.35),('first.last', lambda f, l, d: f"{f}.{l}@{d}" if f and l else None, 0.85),('firstlast', lambda f, l, d: f"{f}{l}@{d}" if f and l else None, 0.75),('first', lambda f, l, d: f"{f}@{d}" if f else None, 0.65),('flast', lambda f, l, d: f"{f[0]}{l}@{d}" if f and l else None, 0.70),('firstl', lambda f, l, d: f"{f}{l[0]}@{d}" if f and l else None, 0.60),('last.first', lambda f, l, d: f"{l}.{f}@{d}" if f and l else None, 0.55),('last', lambda f, l, d: f"{l}@{d}" if l else None, 0.50),('f.last', lambda f, l, d: f"{f[0]}.{l}@{d}" if f and l else None, 0.65),('sales', lambda f, l, d: f"sales@{d}", 0.30),]class PatternGenerator:def generate(self,domain: str,first_name: Optional[str] = None,last_name: Optional[str] = None,

include_generic: bool = True,) -> list[EmailPattern]:if not domain: return []domain = domain.lower().strip()f = self._normalize_name(first_name) if first_name else Nonel = self._normalize_name(last_name) if last_name else Nonepatterns: list[EmailPattern] = []seen: set[str] = set()for name, template_fn, confidence in PATTERN_TEMPLATES:if not include_generic and name in ('info', 'contact', 'hello', 'admin', 'office', 'sales'):continuetry:email = template_fn(f, l, domain)except (TypeError, IndexError):continueif email and email not in seen and self._is_valid_email(email):seen.add(email)patterns.append(EmailPattern(email=email, pattern_name=name, confidence=confidence))patterns.sort(key=lambda p: p.confidence, reverse=True)return patternsdef generate_emails_only(self, domain: str, first_name=None, last_name=None) -> list[str]:return [p.email for p in self.generate(domain, first_name, last_name)] @staticmethoddef _normalize_name(name: str) -> str:name = name.lower().strip()replacements = {'á': 'a', 'é': 'e', 'í': 'i', 'ó': 'o', 'ú': 'u','ñ': 'n', 'ü': 'u', 'ö': 'o', 'ä': 'a'}for accented, plain in replacements.items():name = name.replace(accented, plain)return re.sub(r'[^a-z]', '', name) @staticmethoddef _is_valid_email(email: str) -> bool:return bool(re.match(r'^[a-z0-9._%+\-]+@[a-z0-9.\-]+\.[a-z]{2,}$', email))

5L. discovery/website_crawler.py

discovery/website_crawler.pyfrom __future__ import annotationsimport asyncioimport refrom typing import Optionalfrom urllib.parse import urljoin, urlparseimport aiohttpfrom bs4 import BeautifulSoupfrom loguru import loggerCONTACT_PATHS= ['/contact', '/contact-us', '/contact_us', '/contactus','/about', '/about-us', '/about_us','/team', '/our-team', '/staff','/reach-us', '/get-in-touch','/',] EMAIL_REGEX = re.compile(r'\b[a-zA-Z0-9._%+\-]+@[a-zA-Z0-9.\-]+\.[a-zA-Z]{2,}\b') EXCLUDE_PATTERNS= re.compile(r'(example\.com|test\.com|domain\.com|yoursite|placeholder|' r'sentry|wix\.com|wordpress|squarespace|godaddy)',re.IGNORECASE,)class WebsiteCrawler:def __init__(self, timeout: int = 15, max_pages: int = 5):self.timeout = aiohttp.ClientTimeout(total=timeout)self.max_pages = max_pagesself._session: Optional[aiohttp.ClientSession] = Noneasync def __aenter__(self):self._session = aiohttp.ClientSession(timeout=self.timeout,headers={'User-Agent': 'Mozilla/5.0 (compatible; FirstKnock/1.0)','Accept': 'text/html,application/xhtml+xml','Accept-Language': 'en-US,en;q=0.9',},) return selfasync def __aexit__(self, *args):if self._session:await self._session.close()async def find_emails(self, website_url: str) -> list[str]:if not website_url: return []own_session = Falseif not self._session:self._session = aiohttp.ClientSession(timeout=self.timeout,headers={'User-Agent': 'Mozilla/5.0 (compatible; FirstKnock/1.0)'},)own_session = Truetry:base_url = self._normalize_url(website_url)domain = urlparse(base_url).netloc all_emails: set[str] = set() pages_checked = 0for path in CONTACT_PATHS:if pages_checked >= self.max_pages: breakurl = urljoin(base_url, path)try:emails = await self._extract_emails_from_url(url, domain) all_emails.update(emails)pages_checked += 1if emails and path in ('/contact', '/contact-us', '/about'):breakawait asyncio.sleep(0.5)except Exception as e:logger.debug(f"Crawler error on {url}: {e}")continuereturn self._rank_emails(list(all_emails), domain)finally:if own_session and

self._session: await self._session.close()self._session = Noneasync def _extract_emails_from_url(self, url: str, domain: str) -> list[str]:try:async with self._session.get(url, allow_redirects=True,ssl=False) as resp:if resp.status != 200: return []content_type = resp.headers.get('Content-Type','')if 'text/html' not in content_type: return []html = await resp.text(errors='replace')except (aiohttp.ClientError, asyncio.TimeoutError):return []soup = BeautifulSoup(html, 'lxml')for tag in soup(['script', 'style', 'noscript']):tag.decompose()text_content = soup.get_text(separator=' ')mailto_emails = []for a in soup.find_all('a', href=True):href = a['href']if href.startswith('mailto:'):email = href[7:].split('?')[0].strip()if email: mailto_emails.append(email) text_emails = EMAIL_REGEX.findall(text_content)all_found = set(mailto_emails + text_emails) filtered = []for email in all_found:email = email.lower().strip()if EXCLUDE_PATTERNS.search(email):continueif len(email) > 254: continuefiltered.append(email) return filtered@staticmethoddef _rank_emails(emails: list[str], domain: str) -> list[str]: generic_prefixes = {'info', 'contact', 'hello', 'admin', 'support', 'office'}def score(email: str) -> tuple: local = email.split('@')[0]email_domain = email.split('@')[1] if '@' in email else ''is_domain_match = domain in email_domain or email_domain in domainis_generic = local in generic_prefixesreturn (not is_domain_match, is_generic, local)return sorted(set(emails), key=score)@staticmethoddef _normalize_url(url: str) -> str:if not url.startswith(('http://', 'https://')): url = 'https://' + urlreturn url.rstrip('/')

5M. monitor/app.py

monitor/app.pyfrom __future__ import annotationsfrom flask import Flask, jsonify, render_templatefrom sqlalchemy import Engine, textfrom sqlalchemy.orm import sessionmaker from config.settings import settingsdef create_app(engine: Engine) -> Flask:app = Flask(__name__)SessionLocal = sessionmaker(bind=engine)@app.route('/')def dashboard():return render_template('dashboard.html')@app.route('/api/stats')def api_stats():session = SessionLocal() try:totals = session.execute(text("""SELECTCOUNT(*) AS total_businesses,COUNT(*) FILTER (WHERE email IS NOT NULL) AS with_email,COUNT(*) FILTER (WHERE phone IS NOT NULL) AS with_phone,COUNT(*) FILTER (WHERE email_status = 'valid') AS verified_email,COUNT(*) FILTER (WHERE outreach_status != 'not_contacted') AS contactedFROM businessesWHERE deleted_at IS NULL AND duplicate_status = 'original'""")).mappings().fetchone()by_vertical = session.execute(text("""SELECT vertical, COUNT(*) as cnt, COUNT(email) as with_emailFROM businessesWHERE deleted_at IS NULL AND duplicate_status = 'original'GROUP BY vertical ORDER BY cnt DESC""")).fetchall()jobs = session.execute(text("SELECT status, COUNT(*) as cnt FROM scrape_jobs GROUP BY status")).fetchall()recent = session.execute(text("""SELECT COUNT(*) FROM businessesWHERE created_at > NOW() - INTERVAL '24 hours' AND deleted_at IS NULL""")).scalar()hourly = session.execute(text("""SELECT COUNT(*) FROM businesses WHERE created_at > NOW() - INTERVAL '1 hour' AND deleted_at IS NULL""")).scalar()return jsonify({'totals': dict(totals),'by_vertical': [{'vertical': r[0], 'count': r[1], 'with_email': r[2]}for r in by_vertical],'jobs': {r[0]: r[1] for r in jobs},'recent_24h': recent,'hourly_rate': hourly,})finally:

session.close() @app.route('/api/recent')def api_recent():session = SessionLocal()try:rows = session.execute(text("""SELECT name, vertical, city, state, email, phone, created_atFROM businessesWHERE deleted_at IS NULLORDER BY created_at DESCLIMIT 50""")).fetchall()return jsonify([{'name': r[0], 'vertical': r[1], 'city': r[2],'state': r[3], 'email': r[4], 'phone': r[5],'created_at': r[6].isoformat() if r[6] else None,}for r in rows])finally:session.close()return appif __name__ == '__main__':from sqlalchemy import create_engineeng = create_engine(settings.database_url())app = create_app(eng)app.run(host=settings.MONITOR_HOST,port=settings.MONITOR_PORT, debug=True)

5N. monitor/templates/dashboard.html

The dashboard is a single-page Flask template that auto-refreshes every 10 seconds via JavaScript fetch calls to /api/stats and /api/recent. Key UI components:

- Header: FirstKnock branding, LIVE badge, refresh indicator
- KPI row: Total Businesses, With Email, Verified Email, Hourly Rate
- By Vertical table: vertical name, count, with_email, coverage progress bar
- Job Queue badges: completed (green), pending (gray), running (blue), failed (red), dead (dark red)
- Recent Records table: last 15 businesses with name, vertical, state, email
- Dark theme (#0f1117 background, #1a1d2e cards, #63b3ed accent)

<!DOCTYPE html><html><head><title>FirstKnock — Scraper Dashboard</title><style>* { box-sizing: border-box; margin: 0; padding: 0; }body { font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', sans-serif;background: #0f1117; color: #e2e8f0; min-height: 100vh; }header { background: #1a1d2e; border-bottom: 1px solid #2d3748;padding: 16px 24px; display: flex; align-items: center; gap: 12px; }header h1 { font-size: 1.25rem; font-weight: 700; color: #63b3ed; }.badge { background: #2d3748; padding: 4px 10px; border-radius: 12px;font-size: 0.75rem; color: #a0aec0; }.badge.live { background: #1a3a2a; color: #68d391; }main { padding: 24px; max-width: 1400px; margin: 0 auto; }.grid-4 { display: grid; grid-template-columns: repeat(4, 1fr); gap: 16px; margin-bottom: 24px; }.grid-2 { display: grid; grid-template-columns: 1fr 1fr; gap: 16px; margin-bottom: 24px; }.card { background: #1a1d2e; border: 1px solid #2d3748; border-radius: 12px; padding: 20px; }.card h3 { font-size: 0.75rem; text-transform: uppercase; letter-spacing: 0.05em;color: #718096; margin-bottom: 8px; }.stat-value { font-size: 2rem; font-weight: 700; color: #e2e8f0; }.stat-sub { font-size: 0.8rem; color: #718096; margin-top: 4px; } table { width: 100%; border-collapse: collapse; font-size: 0.85rem; }th { text-align: left; padding: 8px 12px; color: #718096;font-size: 0.75rem; text-transform: uppercase; border-bottom: 1px solid #2d3748; }td { padding: 8px 12px; border-bottom: 1px solid #1e2233; }.progress-bar { background: #2d3748; border-radius: 4px; height: 6px; overflow: hidden; }.progress-fill { height: 100%; background: #63b3ed; border-radius: 4px; }.job-badge { display: inline-block; padding: 2px 8px; border-radius: 10px;font-size: 0.7rem; font-weight: 600; margin: 2px; }.job-badge.completed { background: #1a3a2a; color: #68d391; }.job-badge.pending { background: #2d3748; color: #a0aec0; }.job-badge.running { background: #1a2a3a; color: #63b3ed; }.job-badge.failed {

background: #3a1a1a; color: #fc8181; }.job-badge.dead { background: #2a1a1a; color: #e53e3e; }</style>
</head><body><header><h1> FirstKnock Scraper</h1>
LIVERefreshing every 10s</header>
<main> <h3>Total Businesses</h3><div class="stat-value"
id="stat-total">— — <div
class="card"><h3>With Email</h3> — <div
class="stat-sub" id="stat-email-pct">— <h3>Verified Email</h3><div
class="stat-value" id="stat-verified">— status = valid <div
class="card"><h3>Hourly Rate</h3> — <div
class="stat-sub">records/hour <h3
style="margin-bottom:12px">By Vertical</h3><table id="vertical-table">
<thead><tr><th>Vertical</th><th>Count</th><th>w/ Email</th><th>Coverage</th></tr></thead>
<tbody></tbody></table> <h3
style="margin-bottom:12px">Job Queue</h3>
<h3 style="margin-bottom:12px">Recent Records</h3><table id="recent-table">
<thead><tr><th>Business</th><th>Vertical</th><th>State</th><th>Email</th></tr></thead>
<tbody></tbody></table> </main><script>async function fetchStats() {try {const
[statsRes, recentRes] = await Promise.all([fetch('/api/stats'), fetch('/api/recent')]);const stats = await
statsRes.json();const recent = await recentRes.json();updateDashboard(stats, recent);} catch (e) {
document.getElementById('live-badge').textContent = ' ERROR';}}function fmt(n) { return n == null
? '—' : Number(n).toLocaleString(); }function pct(a, b) { return (!b || b === 0) ? '0%' : (a / b *
100).toFixed(1) + '%'; }function updateDashboard(stats, recent) {const t = stats.totals;
document.getElementById('stat-total').textContent = fmt(t.total_businesses);
document.getElementById('stat-recent').textContent = fmt(stats.recent_24h) + ' added last 24h';
document.getElementById('stat-email').textContent = fmt(t.with_email);
document.getElementById('stat-email-pct').textContent = pct(t.with_email, t.total_businesses) + ' of
total';document.getElementById('stat-verified').textContent = fmt(t.verified_email);
document.getElementById('stat-hourly').textContent = fmt(stats.hourly_rate);const vtbody =
document.querySelector('#vertical-table tbody');vtbody.innerHTML = '';for (const v of
stats.by_vertical) {const coverage = v.count > 0 ? (v.with_email / v.count * 100).toFixed(0) : 0;
vtbody.innerHTML += `<tr><td>${v.vertical.replace('_', ' ')}</td><td>${fmt(v.count)}</td>
<td>${fmt(v.with_email)}</td><td> <div
class="progress-bar" style="flex:1"><div class="progress-fill"
style="width:${coverage}%"></div></div>${coverage}% </td></tr>`;}const jobDiv =
document.getElementById('job-stats');jobDiv.innerHTML = '';for (const [status, count] of
Object.entries(stats.jobs)) {jobDiv.innerHTML += `<span class="job-badge ${status}">${status}:
${fmt(count)}`;}const rtbody = document.querySelector('#recent-table tbody');
rtbody.innerHTML = '';for (const r of recent.slice(0, 15)) {rtbody.innerHTML += `<tr>
<td>${r.name}</td><td>${(r.vertical || '').replace('_', ' ')}</td><td>${r.state || '—'}</td><td>${r.email
|| '—'}</td></tr>`;}}fetchStats();setInterval(fetchStats, 10000);</script></body></html>

Table 9 — Complete Library Reference

Library	Purpose	Install	GitHub
playwright	Browser automation	pip install playwright	github.com/microsoft/playwright-python
patchright	Stealth Playwright fork	pip install patchright	github.com/Kaliiiiiiiiii-Vinyzu/patchright-python
beautifulsoup4	HTML parsing	pip install beautifulsoup4	crummy.com/software/BeautifulSoup/
lxml	Fast HTML parser	pip install lxml	github.com/lxml/lxml
aiohttp	Async HTTP client	pip install aiohttp	github.com/aio-libs/aiohttp
asyncpg	Async PostgreSQL	pip install asyncpg	github.com/MagicStack/asyncpg
dnspython	DNS/MX resolution	pip install dnspython	github.com/rthalley/dnspython
python-whois	WHOIS lookups	pip install python-whois	github.com/richardpenman/whois
phonenumbers	Phone parsing/E.164	pip install phonenumbers	github.com/daviddrysdale/python-phonenumbers
duckduckgo-search	DDG search (no API key)	pip install duckduckgo-search	github.com/deedy5/duckduckgo_search
numpy	Normal dist delays	pip install numpy	github.com/numpy/numpy
pandas	ZIP code loading	pip install pandas	github.com/pandas-dev/pandas
rapidfuzz	Fuzzy deduplication	pip install rapidfuzz	github.com/rapidfuzz/RapidFuzz
2captcha-python	CAPTCHA solving	pip install 2captcha-python	github.com/2captcha/2captcha-python
curl_cffi	TLS fingerprint bypass	pip install curl_cffi	github.com/yifeikong/curl_cffi
loguru	Structured logging	pip install loguru	github.com/Delgan/loguru

sqlalchemy	ORM + query builder	pip install sqlalchemy	github.com/sqlalchemy/sqlalchemy
flask	Monitoring dashboard	pip install flask	github.com/pallets/flask